REFERENCE

Hooks reference

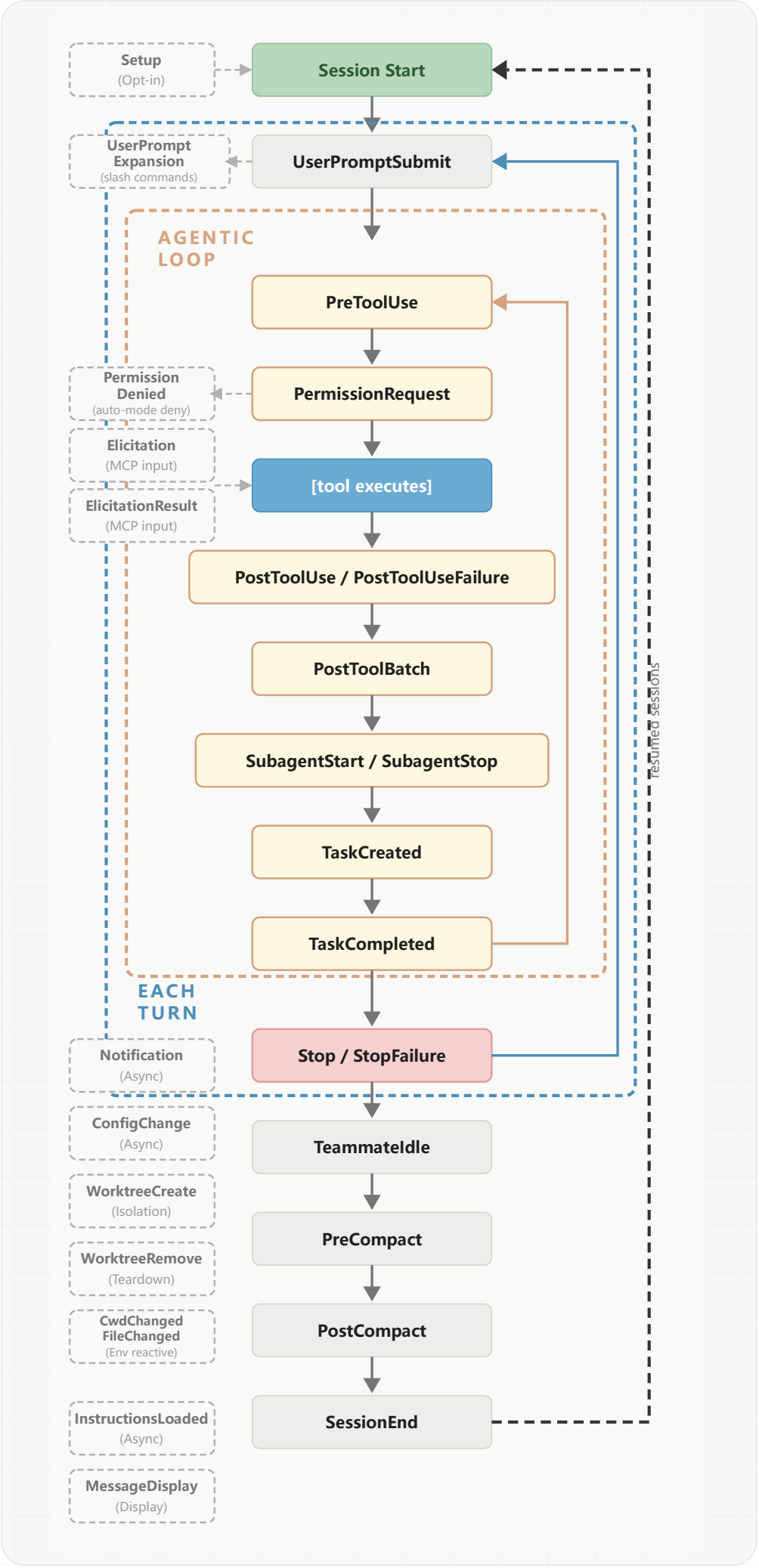
Reference for Claude Code hook events, configuration schema, JSON input/output formats, exit codes, async hooks, HTTP hooks, prompt hooks, and MCP tool hooks.

For a quickstart guide with examples, see **Automate actions with hooks**.

Hooks are user-defined shell commands, HTTP endpoints, or LLM prompts that execute automatically at specific points in Claude Code's lifecycle. Use this reference to look up event schemas, configuration options, JSON input/output formats, and advanced features like async hooks, HTTP hooks, and MCP tool hooks. If you're setting up hooks for the first time, start with the **guide** instead.

Hook lifecycle

Hooks fire at specific points during a Claude Code session. When an event fires and a matcher matches, Claude Code passes JSON context about the event to your hook handler. For command hooks, input arrives on stdin. For HTTP hooks, it arrives as the POST request body. Your handler can then inspect the input, take action, and optionally return a decision. Events fall into three cadences: once per session (`SessionStart` , `SessionEnd`), once per turn (`UserPromptSubmit` , `Stop` , `StopFailure`), and on every tool call inside the agentic loop (`PreToolUse` , `PostToolUse`):



The table below summarizes when each event fires. The **Hook events** section documents the full

input schema and decision control options for each one.

Event	When it fires
`SessionStart`	When a session begins or resumes
`Setup`	When you start Claude Code with `--init-only`, or with `--init` or `--maintenance` in `-p` mode. For one-time preparation in CI or scripts
`UserPromptSubmit`	When you submit a prompt, before Claude processes it
`UserPromptExpansion`	When a user-typed command expands into a prompt, before it reaches Claude. Can block the expansion
`PreToolUse`	Before a tool call executes. Can block it
`PermissionRequest`	When a permission dialog appears
`PermissionDenied`	When a tool call is denied by the auto mode classifier. Return `{retry: true}` to tell the model it may retry the denied tool call
`PostToolUse`	After a tool call succeeds
`PostToolUseFailure`	After a tool call fails
`PostToolBatch`	After a full batch of parallel tool calls resolves, before the next model call
`Notification`	When Claude Code sends a notification
`MessageDisplay`	While assistant message text is displayed
`SubagentStart`	When a subagent is spawned
`SubagentStop`	When a subagent finishes
`TaskCreated`	When a task is being created via `TaskCreate`
`TaskCompleted`	When a task is being marked as completed
`Stop`	When Claude finishes responding
`StopFailure`	When the turn ends due to an API error. Output and exit code are ignored
`TeammateIdle`	When an **agent team** teammate is about to go idle
`InstructionsLoaded`	When a CLAUDE.md or `.claude/rules/*.md` file is loaded into context. Fires at session start and when files are lazily loaded during a session
`ConfigChange`	When a configuration file changes during a session
`CwdChanged`	When the working directory changes, for example when Claude executes a `cd` command. Useful for reactive environment management with tools like direnv
`FileChanged`	When a watched file changes on disk. The `matcher` field specifies which filenames to watch

Event	When it fires
`WorktreeCreate`	When a worktree is being created via `--worktree` or `isolation: "worktree"`. Replaces default git behavior
`WorktreeRemove`	When a worktree is being removed, either at session exit or when a subagent finishes
`PreCompact`	Before context compaction
`PostCompact`	After context compaction completes
`Elicitation`	When an MCP server requests user input during a tool call
`ElicitationResult`	After a user responds to an MCP elicitation, before the response is sent back to the server
`SessionEnd`	When a session terminates

How a hook resolves

To see how these pieces fit together, consider this `PreToolUse` hook that blocks destructive shell commands. The `matcher` narrows to Bash tool calls and the `if` condition narrows further to Bash subcommands matching `rm *`, so `block-rm.sh` only spawns when both filters match:

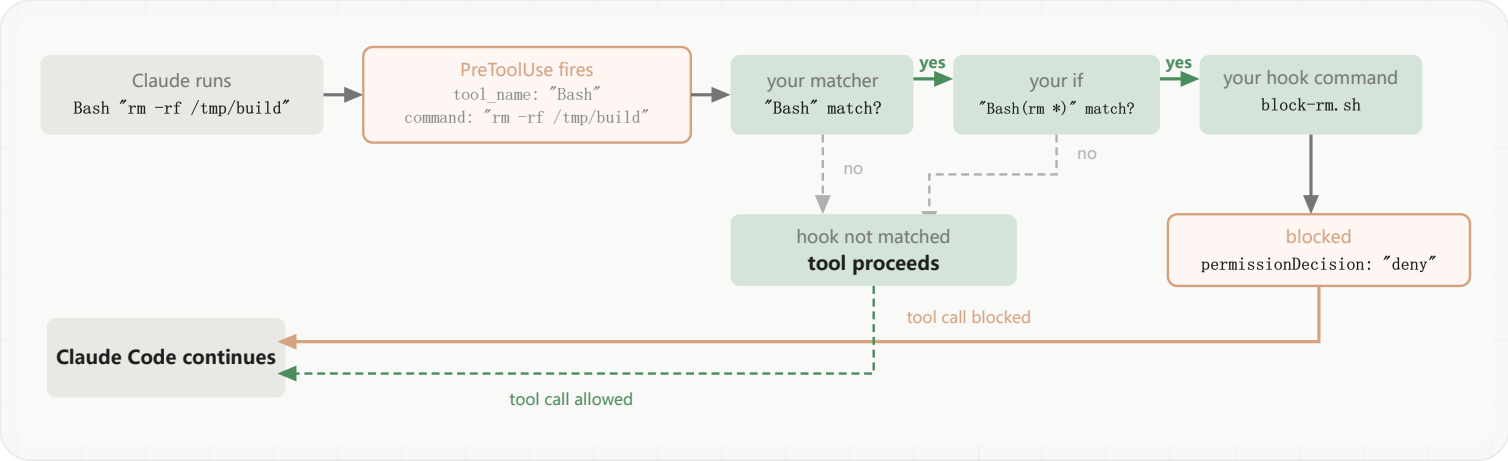
```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "if": "Bash(rm *)",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/block-rm.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

The script reads the JSON input from stdin, extracts the command, and returns a `permissionDecision` of `"deny"` if it contains `rm -rf`:

```
#!/bin/bash
# .claude/hooks/block-rm.sh
COMMAND=$(jq -r '.tool_input.command')

if echo "$COMMAND" | grep -q 'rm -rf'; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "Destructive command blocked by
hook"
    }
  }'
else
  exit 0  # no decision; normal permission flow applies
fi
```

Now suppose Claude Code decides to run `Bash "rm -rf /tmp/build"`. Here's what happens:



1 Event fires

The `PreToolUse` event fires. Claude Code sends the tool input as JSON on stdin to the hook:

```
{ "tool_name": "Bash", "tool_input": { "command": "rm -rf
/tmp/build" }, ... }
```

2 Matcher checks

The matcher `"Bash"` matches the tool name, so this hook group activates. If you omit the

matcher or use `"*"`, the group activates on every occurrence of the event.

3 **If condition checks**

The `if` condition `"Bash(rm *)"` matches because `rm -rf /tmp/build` is a subcommand matching `rm *`, so this handler spawns. If the command had been `npm test`, the `if` check would fail and `block-rm.sh` would never run, avoiding the process spawn overhead. The `if` field is optional; without it, every handler in the matched group runs.

4 **Hook handler runs**

The script inspects the full command and finds `rm -rf`, so it prints a decision to stdout:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Destructive command blocked
by hook"
  }
}
```

If the command had been a safer `rm` variant like `rm file.txt`, the script would hit `exit 0` instead. Exit code 0 with no output means the hook has no decision to report, so the tool call continues through the normal **permission flow**. The hook can deny the call, but staying silent doesn't approve it.

5 **Claude Code acts on the result**

Claude Code reads the JSON decision, blocks the tool call, and shows Claude the reason.

The **Configuration** section below documents the full schema, and each **hook event** section documents what input your command receives and what output it can return.

Configuration

Hooks are defined in JSON settings files. The configuration has three levels of nesting:

1. Choose a **hook event** to respond to, like `PreToolUse` or `Stop`

2. Add a **matcher group** to filter when it fires, like “only for the Bash tool”
3. Define one or more **hook handlers** to run when matched

See **How a hook resolves** above for a complete walkthrough with an annotated example.

> This page uses specific terms for each level: **hook event** for the lifecycle point, **matcher group** for the filter, and **hook handler** for the shell command, HTTP endpoint, MCP tool, prompt, or agent that runs. “Hook” on its own refers to the general feature.

Hook locations

Where you define a hook determines its scope:

Location	Scope	Shareable
`~/.claude/settings.json`	All your projects	No, local to your machine
`.claude/settings.json`	Single project	Yes, can be committed to the repo
`.claude/settings.local.json`	Single project	No, gitignored when Claude Code creates it
Managed policy settings	Organization-wide	Yes, admin-controlled
Plugin `hooks/hooks.json`	When plugin is enabled	Yes, bundled with the plugin
Skill or **agent** frontmatter	While the component is active	Yes, defined in the component file

For details on settings file resolution, see **settings**. Enterprise administrators can use `allowManagedHooksOnly` to block user, project, and plugin hooks. Hooks from plugins force-enabled in managed settings `enabledPlugins` are exempt, so administrators can distribute vetted hooks through an organization marketplace. See **Hook configuration**.

Matcher patterns

The `matcher` field filters when hooks fire. How a matcher is evaluated depends on the characters it contains:

Matcher value	Evaluated as	Example
`"*"`, `""`, or omitted	Match all	fires on every occurrence of the event
Only letters, digits, `_`, spaces, `,`, and `|`	Exact string, or list of exact strings separated by `|` or `,` with optional	`Bash` matches only the Bash tool; `Edit|Write` and `Edit, Write` each

Matcher value	Evaluated as	Example
	surrounding whitespace	match either tool exactly
Contains any other character	JavaScript regular expression	`^Notebook` matches any tool starting with Notebook; `mcp__memory__.*` matches every tool from the `memory` server

Comma separators and the surrounding whitespace tolerance require Claude Code v2.1.191 or later. The `FileChanged` and `StopFailure` events accept only `|` as the list separator and treat `,` as a literal character; all other events listed in the table that follows accept `|` or `,`.

The `FileChanged` event does not follow these rules when building its watch list. See **FileChanged**.

Each event type matches on a different field:

Event	What the matcher filters	Example matcher values
`PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionRequest`, `PermissionDenied`	tool name	`Bash`, `Edit|Write`, `mcp__.*`
`SessionStart`	how the session started	`startup`, `resume`, `clear`, `compact`
`Setup`	which CLI flag triggered setup	`init`, `maintenance`
`SessionEnd`	why the session ended	`clear`, `resume`, `logout`, `prompt_input_exit`, `bypass_permissions_disabled`, `other`
`Notification`	notification type	`permission_prompt`, `idle_prompt`, `auth_success`, `elicitation_dialog`, `elicitation_complete`, `elicitation_response`
`SubagentStart`	agent type	`general-purpose`, `Explore`, `Plan`, or custom agent names
`PreCompact`, `PostCompact`	what triggered compaction	`manual`, `auto`
`SubagentStop`	agent type	same values as `SubagentStart`
`ConfigChange`	configuration source	`user_settings`, `project_settings`, `local_settings`, `policy_settings`, `skills`
`CwdChanged`	no matcher support	always fires on every directory change

Event	What the matcher filters	Example matcher values
`FileChanged`	literal filenames to watch (see **FileChanged**)	`.envrc|.env`
`StopFailure`	error type	`rate_limit`, `overloaded`, `authentication_failed`, `oauth_org_not_allowed`, `billing_error`, `invalid_request`, `model_not_found`, `server_error`, `max_output_tokens`, `unknown`
`InstructionsLoaded`	load reason	`session_start`, `nested_traversal`, `path_glob_match`, `include`, `compact`
`UserPromptExpansion`	command name	your skill or command names
`Elicitation`	MCP server name	your configured MCP server names
`ElicitationResult`	MCP server name	same values as `Elicitation`
`UserPromptSubmit`, `PostToolBatch`, `Stop`, `TeammateIdle`, `TaskCreated`, `TaskCompleted`, `WorktreeCreate`, `WorktreeRemove`, `MessageDisplay`	no matcher support	always fires on every occurrence

The matcher runs against a field from the **JSON input** that Claude Code sends to your hook on stdin. For tool events, that field is `tool_name`. Each **hook event** section lists the full set of matcher values and the input schema for that event.

This example runs a linting script only when Claude writes or edits a file:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/lint-check.sh"
          }
        ]
      }
    ]
  }
}
```

`UserPromptSubmit`, `PostToolBatch`, `Stop`, `TeammateIdle`, `TaskCreated`, `TaskCompleted`, `WorktreeCreate`, `WorktreeRemove`, and `CwdChanged` don't support matchers and always fire on every occurrence. If you add a `matcher` field to these events, it is silently ignored.

For tool events, you can filter more narrowly by setting the **`if` field** on individual hook handlers. `if` uses **permission rule syntax** to match against the tool name and arguments together, so `"Bash(git *)"` runs when any subcommand of the Bash input matches `git *` and `"Edit(*.ts)"` runs only for TypeScript files.

Match MCP tools

MCP server tools appear as regular tools in tool events (`PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionRequest`, `PermissionDenied`), so you can match them the same way you match any other tool name.

MCP tools follow the naming pattern `mcp__<server>__<tool>`, for example:

- `mcp__memory__create_entities`: Memory server's create entities tool
- `mcp__filesystem__read_file`: Filesystem server's read file tool
- `mcp__github__search_repositories`: GitHub server's search tool

To match every tool from a server, append `.*` to the server prefix. The `.*` is required: a matcher like `mcp__memory` contains only letters and underscores, so it is compared as an exact string and matches no tool.

- `mcp__memory__.*` matches all tools from the `memory` server
- `mcp__.*__write.*` matches any tool whose name starts with `write` from any server

This example logs all memory server operations and validates write operations from any MCP server:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "mcp__memory__.*",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Memory operation initiated' >>
~/mcp-operations.log"
          }
        ]
      },
      {
        "matcher": "mcp__.*__write.*",
        "hooks": [
          {
            "type": "command",
            "command": "/home/user/scripts/validate-mcp-write.py"
          }
        ]
      }
    ]
  }
}
```

Hook handler fields

Each object in the inner `hooks` array is a hook handler: the shell command, HTTP endpoint, MCP tool, LLM prompt, or agent that runs when the matcher matches. There are five types:

- **Command hooks** (`type: "command"`): run a shell command. Your script receives the event's **JSON input** on stdin and communicates results back through exit codes and stdout.
- **HTTP hooks** (`type: "http"`): send the event's JSON input as an HTTP POST request to a URL. The endpoint communicates results back through the response body using the same **JSON output format** as command hooks.

- **MCP tool hooks** (`type: "mcp_tool"`): call a tool on an already-connected **MCP server**. The tool's text output is treated like command-hook stdout.
- **Prompt hooks** (`type: "prompt"`): send a prompt to a Claude model for single-turn evaluation. The model returns a yes/no decision as JSON. See **Prompt-based hooks**.
- **Agent hooks** (`type: "agent"`): spawn a subagent that can use tools like Read, Grep, and Glob to verify conditions before returning a decision. Agent hooks are experimental and may change. See **Agent-based hooks**.

All matching hooks run in parallel, and identical handlers are deduplicated automatically. Command hooks are deduplicated by command string and `args`, and HTTP hooks are deduplicated by URL.

Handlers run in the current directory with Claude Code's environment. The `$CLAUDE_CODE_REMOTE` environment variable is set to `"true"` in remote web environments and not set in the local CLI.

Common fields

These fields apply to all hook types:

Field	Required	Description
`type`	yes	`"command"`, `"http"`, `"mcp_tool"`, `"prompt"`, or `"agent"`
`if`	no	Permission rule syntax to filter when this hook runs, such as `"Bash(git *)"` or `"Edit(*.ts)"`. The hook command only runs if the tool call matches the pattern. See the **Bash matching table** below for how Bash patterns evaluate against subcommands, `$()`, and backticks. Only evaluated on tool events: `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionRequest`, and `PermissionDenied`. On other events, a hook with `if` set never runs. Uses the same syntax as **permission rules**
`timeout`	no	Seconds before canceling. Defaults: 600 for `command`, `http`, and `mcp_tool`; 30 for `prompt`; 60 for `agent`. `UserPromptSubmit` lowers the `command`, `http`, and `mcp_tool` default to 30, and `MessageDisplay` lowers it to 10
`statusMessage`	no	Custom spinner message displayed while the hook runs
`once`	no	If `true`, runs once per session then is removed. Only honored for hooks declared in **skill frontmatter**; ignored in settings files and agent frontmatter

The `if` field holds exactly one permission rule. There is no `&&`, `||`, or list syntax for combining rules; to apply multiple conditions, define a separate hook handler for each.

For Bash patterns, whether your hook command runs depends on the shape of the pattern and the Bash command Claude is invoking. Leading `VAR=value` assignments are stripped before matching.

`if` pattern	Bash command	Hook runs?	Why
`Bash(git *)`	`FOO=bar git push`	yes	leading assignments are stripped; `git push` matches
`Bash(git *)`	`npm test && git push`	yes	each subcommand is checked; `git push` matches
`Bash(rm *)`	`echo $(rm -rf /)`	yes	commands inside `$()` and backticks are checked; `rm -rf /` matches
`Bash(rm *)`	`echo $(date)`	no	no subcommand matches `rm *`
`Bash(git push *)`	`echo $(date)`	yes	patterns that specify more than the command name run the hook anyway on `$()`, backticks, or `$VAR`

The filter also fails open, running your hook regardless of pattern, when the Bash command cannot be parsed. Because the `if` filter is best-effort, use the **permission system** rather than a hook to enforce a hard allow or deny.

Command hook fields

In addition to the **common fields**, command hooks accept these fields:

Field	Required	Description
`command`	yes	Shell command to execute. With `args`, the executable to spawn directly. See **Exec form and shell form**
`args`	no	Argument list. When present, `command` is resolved as an executable and spawned directly with `args` as the argument vector, with no shell involved. See **Exec form and shell form**
`async`	no	If `true`, runs in the background without blocking. See **Run hooks in the background**
`asyncRewake`	no	If `true`, runs in the background and wakes Claude on exit code 2. Implies `async`. The hook's stderr, or stdout if stderr is empty, is shown to Claude as a system reminder so it can react to a long-running background failure
`shell`	no	Shell to use for this hook. Accepts `"bash"` (default) or `"powershell"`. Setting `"powershell"` runs the command via PowerShell on Windows. Does not require `CLAUDE_CODE_USE_POWERSHELL_TOOL` since hooks spawn PowerShell

Field	Required	Description
		directly. Ignored when `args` is set

Exec form and shell form

A command hook runs as exec form when `args` is set, and shell form when `args` is omitted. Set `args` whenever the hook references a **path placeholder**, since each element is passed as one argument with no quoting. Omit `args` when you need shell features like pipes or `&&`, or when neither concern applies.

Exec form runs when `args` is present. Claude Code resolves `command` as an executable on `PATH` and spawns it directly with `args` as the argument vector. There is no shell, so each `args` element is one argument exactly as written, and path placeholders like `${CLAUDE_PLUGIN_ROOT}` are substituted into `command` and into each `args` element as plain strings. Special characters such as apostrophes, `$`, and backticks pass through verbatim because there is no shell to interpret them. No shell tokenization happens on any platform.

Shell form runs when `args` is absent. The `command` string is passed to a shell: `sh -c` on macOS and Linux, Git Bash on Windows, or PowerShell when Git Bash isn't installed. Set the `shell` field to choose explicitly. The shell tokenizes the string, expands variables, and interprets pipes, `&&`, redirects, and globs.

> On Windows, exec form requires `command` to resolve to a real executable such as a `.exe`. The `.cmd` and `.bat` shims that npm, npx, eslint, and other tools install in `node_modules/.bin` are not executables and cannot be spawned without a shell. To run them in exec form, invoke the underlying script with `node` directly, for example `"command": "node", "args": ["${CLAUDE_PLUGIN_ROOT}/node_modules/eslint/bin/eslint.js"]`. The `node` plus script-path pattern works on every platform because `node.exe` is a real binary. To run a `.cmd` or `.bat` shim by name, use shell form.

This example runs a Node script bundled with a plugin. Exec form passes the resolved script path as one argument with no quoting:

```
{
  "type": "command",
  "command": "node",
  "args": ["${CLAUDE_PLUGIN_ROOT}/scripts/format.js", "--fix"]
}
```

The equivalent shell form needs quoting to handle paths with spaces or special characters:

```
{
  "type": "command",
  "command": "node \"${CLAUDE_PLUGIN_ROOT}\"/scripts/format.js --
fix"
}
```

Both forms support the same **path placeholders**, and both export them as the environment variables `CLAUDE_PROJECT_DIR`, `CLAUDE_PLUGIN_ROOT`, and `CLAUDE_PLUGIN_DATA` on the spawned process, so a script can read `process.env.CLAUDE_PLUGIN_ROOT` regardless of how it was launched. Plugin hooks additionally substitute `${user_config.*}` values; see **User configuration**.

> In exec form, `command` is the executable name or path only. If `command` is a bare name with no path separator and contains whitespace alongside `args`, Claude Code logs a warning because the spawn will fail: there is no executable named `node script.js`. Move the extra tokens into `args`. Absolute paths with spaces, such as `C:\Program Files\nodejs\node.exe`, are a single valid executable and do not trigger the warning.

HTTP hook fields

In addition to the **common fields**, HTTP hooks accept these fields:

Field	Required	Description
`url`	yes	URL to send the POST request to
`headers`	no	Additional HTTP headers as key-value pairs. Values support environment variable interpolation using `$VAR_NAME` or `${VAR_NAME}` syntax. Only variables listed in `allowedEnvVars` are resolved
`allowedEnvVars`	no	List of environment variable names that may be interpolated into header values. References to unlisted variables are replaced with empty strings. Required for any env var interpolation to work

Claude Code sends the hook's **JSON input** as the POST request body with `Content-Type: application/json`. The response body uses the same **JSON output format** as command hooks.

Error handling differs from command hooks: non-2xx responses, connection failures, and timeouts all produce non-blocking errors that allow execution to continue. To block a tool call or deny a permission, return a 2xx response with a JSON body containing `decision: "block"` or a `hookSpecificOutput` with `permissionDecision: "deny"`.

This example sends `PreToolUse` events to a local validation service, authenticating with a token from the `MY_TOKEN` environment variable:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "http",
            "url": "http://localhost:8080/hooks/pre-tool-use",
            "timeout": 30,
            "headers": {
              "Authorization": "Bearer $MY_TOKEN"
            },
            "allowedEnvVars": ["MY_TOKEN"]
          }
        ]
      }
    ]
  }
}
```

MCP tool hook fields

In addition to the **common fields**, MCP tool hooks accept these fields:

Field	Required	Description
`server`	yes	Name of a configured MCP server. The server must already be connected; the hook never triggers an OAuth or connection flow
`tool`	yes	Name of the tool to call on that server
`input`	no	Arguments passed to the tool. String values support `${path}` substitution from the hook's **JSON input**, such as `"${tool_input.file_path}"`

The tool's text content is treated like command-hook stdout: if it parses as valid **JSON output** it is processed as a decision, otherwise it is shown as plain text. If the named server is not connected, or the tool returns `isError: true`, the hook produces a non-blocking error and execution continues.

MCP tool hooks are available on every hook event once Claude Code has connected to your MCP servers. `SessionStart` and `Setup` typically fire before servers finish connecting, so hooks on those events should expect the "not connected" error on first run.

This example calls the `security_scan` tool on the `my_server` MCP server after each `Write` or `Edit`, passing the edited file's path:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "mcp_tool",
            "server": "my_server",
            "tool": "security_scan",
            "input": { "file_path": "${tool_input.file_path}" }
          }
        ]
      }
    ]
  }
}
```

Prompt and agent hook fields

In addition to the **common fields**, prompt and agent hooks accept these fields:

Field	Required	Description
`prompt`	yes	Prompt text to send to the model. Use `$ARGUMENTS` as a placeholder for the hook input JSON. Escape with a backslash to include literal text: `\$1.00` renders as `$1.00`
`model`	no	Model to use for evaluation. Defaults to a fast model

Reference scripts by path

Use these placeholders to reference hook scripts relative to the project or plugin root, regardless of the working directory when the hook runs:

- `${CLAUDE_PROJECT_DIR}` : the project root. Claude Code also sets this variable in the environment of **stdio MCP servers** and plugin LSP servers.
- `${CLAUDE_PLUGIN_ROOT}` : the plugin's installation directory, for scripts bundled with a **plugin**. Changes on each plugin update.
- `${CLAUDE_PLUGIN_DATA}` : the plugin's **persistent data directory**, for dependencies and state that should survive plugin updates.

Prefer **exec form** for any hook that references a path placeholder. Exec form passes each `args` element as one argument with no shell tokenization, so paths with spaces or special characters need no quoting. In shell form, wrap each placeholder in double quotes.

Project scripts

This example uses `${CLAUDE_PROJECT_DIR}` to run a style checker from the project's `.claude/hooks/` directory after any `Write` or `Edit` tool call:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/check-style.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

Plugin scripts

Define plugin hooks in `hooks/hooks.json` with an optional top-level `description` field. When a plugin is enabled, its hooks merge with your user and project hooks.This example runs a formatting

script bundled with the plugin:

```
{
  "description": "Automatic code formatting",
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PLUGIN_ROOT}/scripts/format.sh",
            "args": [],
            "timeout": 30
          }
        ]
      }
    ]
  }
}
```

See the **plugin components reference** for details on creating plugin hooks.

Used by `UserPromptSubmit`, `UserPromptExpansion`, `PostToolUse`, `PostToolUseFailure`, `PostToolBatch`, `Stop`, `SubagentStop`, `ConfigChange`, and `PreCompact`. The only value is `"block"`. To allow the action to proceed, omit `decision` from your JSON, or exit 0 without any JSON at all:

```
{
  "decision": "block",
  "reason": "Test suite must pass before proceeding"
}
```

Uses `hookSpecificOutput` for richer control: allow, deny, or escalate to the user. You can also modify tool input before it runs or inject additional context for Claude. See **PreToolUse decision**

control for the full set of options.

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Database writes are not allowed"
  }
}
```

Uses `hookSpecificOutput` to allow or deny a permission request on behalf of the user. When allowing, you can also modify the tool's input or apply permission rules so the user isn't prompted again. See **PermissionRequest decision control** for the full set of options.

```
{
  "hookSpecificOutput": {
    "hookEventName": "PermissionRequest",
    "decision": {
      "behavior": "allow",
      "updatedInput": {
        "command": "npm run lint"
      }
    }
  }
}
```

Register a command hook for the event in your settings file:

```
{
  "hooks": {
    "MessageDisplay": [
      {
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/plain-display.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

Save this script to `.claude/hooks/plain-display.sh` in your project and make it executable with `chmod +x`:

```
#!/bin/bash
jq '{hookSpecificOutput: {hookEventName: "MessageDisplay",
displayContent: (.delta | gsub("\\*\\*"; "") | gsub("`"; ""))}}'
```

The script needs `jq` on your `PATH`.

Register a command hook that runs the script through PowerShell:

```
{
  "hooks": {
    "MessageDisplay": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "powershell.exe",
            "args": [
              "-NoProfile",
              "-ExecutionPolicy",
              "Bypass",
              "-File",
              "${CLAUDE_PROJECT_DIR}/.claude/hooks/plain-
display.ps1"
            ]
          }
        ]
      }
    ]
  }
}
```

The `-NoProfile` flag skips loading your PowerShell profile so the hook starts fast, and `-ExecutionPolicy Bypass` lets PowerShell run the local script file.Save this script to `.claude/hooks/plain-display.ps1` in your project:

```
$batch = [Console]::In.ReadToEnd() | ConvertFrom-Json
$text = $batch.delta -replace '\*\*', '' -replace '``', ''
@{
  hookSpecificOutput = @{
    hookEventName = "MessageDisplay"
    displayContent = $text
  }
} | ConvertTo-Json
```

Hooks in skills and agents

In addition to settings files and plugins, hooks can be defined directly in **skills** and **subagents** using

frontmatter. These hooks are scoped to the component's lifecycle and only run when that component is active.

All hook events are supported. For subagents, `Stop` hooks are automatically converted to `SubagentStop` since that is the event that fires when a subagent completes.

Hooks use the same configuration format as settings-based hooks but are scoped to the component's lifetime and cleaned up when it finishes.

This skill defines a `PreToolUse` hook that runs a security validation script before each `Bash` command:

```
---
name: secure-operations
description: Perform operations with security checks
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/security-check.sh"
---
```

Agents use the same format in their YAML frontmatter.

The `/hooks` menu

Type `/hooks` in Claude Code to open a read-only browser for your configured hooks. The menu shows every hook event with a count of configured hooks, lets you drill into matchers, and shows the full details of each hook handler. Use it to verify configuration, check which settings file a hook came from, or inspect a hook's command, prompt, or URL.

The menu displays all five hook types: `command`, `prompt`, `agent`, `http`, and `mcp_tool`. Each hook is labeled with a `[type]` prefix and a source indicating where it was defined:

- `User`: from `~/.claude/settings.json`
- `Project`: from `.claude/settings.json`
- `Local`: from `.claude/settings.local.json`
- `Plugin`: from a plugin's `hooks/hooks.json`

- `Session`: registered in memory for the current session
- `Built-in`: registered internally by Claude Code

Selecting a hook opens a detail view showing its event, matcher, type, source file, and the full command, prompt, or URL. The menu is read-only: to add, modify, or remove hooks, edit the settings JSON directly or ask Claude to make the change.

Disable or remove hooks

To remove a hook, delete its entry from the settings JSON file.

To temporarily disable all hooks without removing them, set `"disableAllHooks": true` in your settings file. There is no way to disable an individual hook while keeping it in the configuration.

The `disableAllHooks` setting respects the managed settings hierarchy. If an administrator has configured hooks through managed policy settings, `disableAllHooks` set in user, project, or local settings cannot disable those managed hooks. Only `disableAllHooks` set at the managed settings level can disable managed hooks.

Direct edits to hooks in settings files are normally picked up automatically by the file watcher.

Hook input and output

Command hooks receive JSON data via stdin and communicate results through exit codes, stdout, and stderr. HTTP hooks receive the same JSON as the POST request body and communicate results through the HTTP response body. This section covers fields and behavior common to all events. Each event's section under **Hook events** includes its specific input schema and decision control options.

On macOS and Linux, command hooks run in their own session without a controlling terminal as of v2.1.139. The hook process and any child processes cannot open `/dev/tty` or send escape sequences directly to the Claude Code interface. Windows has no `/dev/tty`. To surface a message to the user on any platform, return `systemMessage` in JSON output. To trigger a desktop notification, set a window title, or ring the bell, return `terminalSequence` instead.

Common input fields

Hook events receive these fields as JSON, in addition to event-specific fields documented in each **hook event** section. For command hooks, this JSON arrives via stdin. For HTTP hooks, it arrives as the POST request body.

Field	Description
`session_id`	Current session identifier
`transcript_path`	Path to conversation JSON
`cwd`	Current working directory when the hook is invoked
`permission_mode`	Current **permission mode**: `"default"`, `"plan"`, `"acceptEdits"`, `"auto"`, `"dontAsk"`, or `"bypassPermissions"`. Not all events receive this field: see each event's JSON example below to check
`effort`	Object with a `level` field holding the active **effort level** for the turn: `"low"`, `"medium"`, `"high"`, `"xhigh"`, or `"max"`. If the requested model effort exceeds what the current model supports, this is the downgraded level the model actually used. Ultracode is not a distinct level and reports as `"xhigh"`. The object matches the **status line** `effort` field. Present for events that fire within a tool-use context, such as `PreToolUse`, `PostToolUse`, `Stop`, and `SubagentStop`, when the current model supports the effort parameter. The level is also available to hook commands and the Bash tool as the `$CLAUDE_EFFORT` environment variable.
`hook_event_name`	Name of the event that fired

When running with `--agent` or inside a subagent, two additional fields are included:

Field	Description
`agent_id`	Unique identifier for the subagent. Present only when the hook fires inside a subagent call. Use this to distinguish subagent hook calls from main-thread calls.
`agent_type`	Agent name (for example, `"Explore"` or `"security-reviewer"`). Present when the session uses `--agent` or the hook fires inside a subagent. For subagents, the subagent's type takes precedence over the session's `--agent` value. For **custom subagents**, this is the `name` field from the agent's frontmatter, not the filename.

Only `SessionStart` hooks can receive a `model` field, and it is not guaranteed to be present. There is no `$CLAUDE_MODEL` environment variable. A hook process inherits the parent environment, so it can read `$ANTHROPIC_MODEL` if you set it in your shell, but that value does not change when you switch models with `/model` during a session.

For example, a `PreToolUse` hook for a Bash command receives this on stdin:

```
{
  "session_id": "abc123",
  "transcript_path":
"/home/user/.claude/projects/.../transcript.jsonl",
  "cwd": "/home/user/my-project",
  "permission_mode": "default",
  "hook_event_name": "PreToolUse",
  "tool_name": "Bash",
  "tool_input": {
    "command": "npm test"
  }
}
```

The `tool_name` and `tool_input` fields are event-specific. Each **hook event** section documents the additional fields for that event.

Exit code output

The exit code from your hook command tells Claude Code whether the action should proceed, be blocked, or be ignored.

Exit 0 means success. Claude Code parses stdout for **JSON output fields**. JSON output is only processed on exit 0. For most events, stdout is written to the debug log but not shown in the transcript. The exceptions are `UserPromptSubmit`, `UserPromptExpansion`, and `SessionStart`, where stdout is added as context that Claude can see and act on.

Exit 2 means a blocking error. Claude Code ignores stdout and any JSON in it. Instead, stderr text is fed back to Claude as an error message. The effect depends on the event: `PreToolUse` blocks the tool call, `UserPromptSubmit` rejects the prompt, and so on. See **exit code 2 behavior** for the full list.

Any other exit code is a non-blocking error for most hook events. The transcript shows a `<hook name> hook error` notice followed by the first line of stderr, so you can identify the cause without `--debug`. Execution continues and the full stderr is written to the debug log.

For example, a hook command script that blocks dangerous Bash commands:

```
#!/bin/bash
# Reads JSON input from stdin, checks the command
command=$(jq -r '.tool_input.command' < /dev/stdin)

if [[ "$command" == rm* ]]; then
  echo "Blocked: rm commands are not allowed" >&2
  exit 2  # Blocking error: tool call is prevented
fi

exit 0  # No decision: the normal permission flow applies
```

⚠ For most hook events, only exit code 2 blocks the action. Claude Code treats exit code 1 as a non-blocking error and proceeds with the action, even though 1 is the conventional Unix failure code. If your hook is meant to enforce a policy, use `exit 2`. The exception is `WorktreeCreate`, where any non-zero exit code aborts worktree creation.

Exit code 2 behavior per event

Exit code 2 is the way a hook signals "stop, don't do this." The effect depends on the event, because some events represent actions that can be blocked (like a tool call that hasn't happened yet) and others represent things that already happened or can't be prevented.

Hook event	Can block?	What happens on exit 2
`PreToolUse`	Yes	Blocks the tool call
`PermissionRequest`	Yes	Denies the permission
`UserPromptSubmit`	Yes	Blocks prompt processing and erases the prompt
`UserPromptExpansion`	Yes	Blocks the expansion
`Stop`	Yes	Prevents Claude from stopping, continues the conversation
`SubagentStop`	Yes	Prevents the subagent from stopping
`TeammateIdle`	Yes	Prevents the teammate from going idle (teammate continues working)
`TaskCreated`	Yes	Rolls back the task creation
`TaskCompleted`	Yes	Prevents the task from being marked as completed
`ConfigChange`	Yes	Blocks the configuration change from taking effect (except `policy_settings`)

Hook event	Can block?	What happens on exit 2
`StopFailure`	No	Output and exit code are ignored
`PostToolUse`	No	Shows stderr to Claude (tool already ran)
`PostToolUseFailure`	No	Shows stderr to Claude (tool already failed)
`PostToolBatch`	Yes	Stops the agentic loop before the next model call
`PermissionDenied`	No	Exit code and stderr are ignored (denial already occurred). Use JSON `hookSpecificOutput.retry: true` to tell the model it may retry
`Notification`	No	Shows stderr to user only
`SubagentStart`	No	Shows stderr to user only
`SessionStart`	No	Shows stderr to user only
`Setup`	No	Shows stderr to user only
`SessionEnd`	No	Shows stderr to user only
`CwdChanged`	No	Shows stderr to user only
`FileChanged`	No	Shows stderr to user only
`PreCompact`	Yes	Blocks compaction
`PostCompact`	No	Shows stderr to user only
`Elicitation`	Yes	Denies the elicitation
`ElicitationResult`	Yes	Blocks the response (action becomes decline)
`WorktreeCreate`	Yes	Any non-zero exit code causes worktree creation to fail
`WorktreeRemove`	No	Failures are logged in debug mode only
`InstructionsLoaded`	No	Exit code is ignored
`MessageDisplay`	No	The original text is displayed

HTTP response handling

HTTP hooks use HTTP status codes and response bodies instead of exit codes and stdout:

- **2xx with an empty body**: success, equivalent to exit code 0 with no output
- **2xx with a plain text body**: success, the text is added as context
- **2xx with a JSON body**: success, parsed using the same **JSON output** schema as command

hooks

- **Non-2xx status**: non-blocking error, execution continues
- **Connection failure or timeout**: non-blocking error, execution continues

Unlike command hooks, HTTP hooks cannot signal a blocking error through status codes alone. To block a tool call or deny a permission, return a 2xx response with a JSON body containing the appropriate decision fields.

JSON output

Exit codes only let you block or stay silent, but JSON output gives you finer-grained control. Instead of exiting with code 2 to block, exit 0 and print a JSON object to stdout. Claude Code reads specific fields from that JSON to control behavior, including **decision control** for blocking, allowing, or escalating to the user.

> You must choose one approach per hook, not both: either use exit codes alone for signaling, or exit 0 and print JSON for structured control. Claude Code only processes JSON on exit 0. If you exit 2, any JSON is ignored.

Your hook's stdout must contain only the JSON object. If your shell profile prints text on startup, it can interfere with JSON parsing. See **JSON validation failed** in the troubleshooting guide.

Hook output strings, including `additionalContext`, `systemMessage`, and plain stdout, are capped at 10,000 characters. Output that exceeds this limit is saved to a file and replaced with a preview and file path, the same way large tool results are handled.

The JSON object supports three kinds of fields:

- **Universal fields** like `continue` work across all events. These are listed in the table below.
- **Top-level** `decision` **and** `reason` are used by some events to block or provide feedback.
- `hookSpecificOutput` is a nested object for events that need richer control. It requires a `hookEventName` field set to the event name.

Field	Default	Description
`continue`	`true`	If `false`, Claude stops processing entirely after the hook runs. Takes precedence over any event-specific decision fields
`stopReason`	none	Message shown to the user when `continue` is `false`. Not shown to Claude

Field	Default	Description
`suppressOutput`	`false`	If `true`, hides the hook's stdout from the transcript. Stdout still appears in the debug log
`systemMessage`	none	Warning message shown to the user
`terminalSequence`	none	A terminal escape sequence for Claude Code to emit on your behalf, such as a desktop notification, window title, or bell. Restricted to OSC `0` / `1` / `2` / `9` / `99` / `777` and BEL. If the value contains anything outside the allowlist, the field is ignored. Use this instead of writing to `/dev/tty`, which is unavailable to hooks

To stop Claude entirely regardless of event type:

```
{ "continue": false, "stopReason": "Build failed, fix errors
before continuing" }
```

Emit terminal notifications

The `terminalSequence` field requires Claude Code v2.1.141 or later.

Hooks run without a controlling terminal, so writing escape sequences directly to `/dev/tty` fails. Instead, return the escape sequence in the `terminalSequence` field and Claude Code emits it for you through its own terminal write path. This is race-free, works inside tmux and GNU screen, and works on Windows where there is no `/dev/tty`.

The field accepts a string of one or more allowlisted escape sequences:

- OSC `0`, `1`, `2`: window and icon titles
- OSC `9`: iTerm2, ConEmu, Windows Terminal, and WezTerm notifications, including `9;4` taskbar progress
- OSC `99`: Kitty notifications
- OSC `777`: urxvt, Ghostty, and Warp notifications
- Bare BEL

Sequences may be terminated with BEL or with ST. Anything outside the allowlist, including CSI cursor and color sequences, OSC palette sequences, OSC 8 hyperlinks, OSC 52 clipboard writes, and OSC 1337, is rejected and the field is ignored.

The example below fires a desktop notification from a `Notification` hook. The escape sequence is built with `printf` octal escapes so the control bytes never appear on the shell command line, and

`jq -n --arg` builds the JSON output so quotes, backslashes, and newlines in the notification message are escaped correctly:

```
#!/bin/bash
# Notification hook: ping the desktop when Claude Code needs
attention.
input=$(cat)
title="Claude Code"
body=$(jq -r '.message // "Needs your attention"' <<<"$input")
seq=$(printf '\033]777;notify;%s;%s\007' "$title" "$body")
jq -nc --arg seq "$seq" '{terminalSequence: $seq}'
```

The `{ "terminalSequence": "..." }` shape is the same from any shell or language. On Windows, build the escape string in PowerShell or a script and emit the same JSON object.

> `terminalSequence` is the supported replacement for hooks that previously wrote escape sequences directly to `/dev/tty`. The allowlist is restricted to sequences that cannot move the cursor or alter colors, so a hook can never corrupt an on-screen prompt.

Add context for Claude

The `additionalContext` field passes a string from your hook into Claude's context window. Claude Code wraps the string in a system reminder and inserts it into the conversation at the point where the hook fired. Claude reads the reminder on the next model request, but it does not appear as a chat message in the interface.

Return `additionalContext` inside `hookSpecificOutput` alongside the event name:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PostToolUse",
    "additionalContext": "This file is generated. Edit
src/schema.ts and run `bun generate` instead."
  }
}
```

Where the reminder appears depends on the event:

- **SessionStart**, **Setup**, and **SubagentStart**: at the start of the conversation, before the first prompt

- **UserPromptSubmit** and **UserPromptExpansion**: alongside the submitted prompt
- **PreToolUse**, **PostToolUse**, **PostToolUseFailure**, and **PostToolBatch**: next to the tool result
- **Stop** and **SubagentStop**: at the end of the turn. The conversation continues so Claude can act on the feedback. See **Stop decision control**

When several hooks return `additionalContext` for the same event, Claude receives all of the values. If a value exceeds 10,000 characters, Claude Code writes the full text to a file in the session directory and passes Claude the file path with a short preview instead.

Use `additionalContext` for information Claude should know about the current state of your environment or the operation that just ran:

- **Environment state**: the current branch, deployment target, or active feature flags
- **Conditional project rules**: which test command applies to the file just edited, which directories are read-only in this worktree
- **External data**: open issues assigned to you, recent CI results, content fetched from an internal service

For instructions that never change, prefer **CLAUDE.md**. It loads without running a script and is the standard place for static project conventions.

Write the text as factual statements rather than imperative system instructions. Phrasing such as "The deployment target is production" or "This repo uses `bun test`" reads as project information. Text framed as out-of-band system commands can trigger Claude's prompt-injection defenses, which causes Claude to surface the text to you instead of treating it as context.

Once injected, the text is saved in the session transcript. For mid-session events like `PostToolUse` or `UserPromptSubmit`, resuming with `--continue` or `--resume` replays the saved text rather than re-running the hook for past turns, so values like timestamps or commit SHAs become stale on resume. `SessionStart` hooks run again on resume with `source` set to `"resume"`, so they can refresh their context.

Decision control

Not every event supports blocking or controlling behavior through JSON. The events that do each use a different set of fields to express that decision. Use this table as a quick reference before writing a hook:

Events	Decision pattern	Key fields
UserPromptSubmit, UserPromptExpansion, PostToolUse, PostToolUseFailure, PostToolBatch, Stop, SubagentStop, ConfigChange, PreCompact	Top-level `decision`	`decision: "block"`, `reason`. Stop and SubagentStop also accept `hookSpecificOutput.additionalContext` for **non-error feedback that continues the conversation**
TeammateIdle, TaskCreated, TaskCompleted	Exit code or `continue: false`	Exit code 2 blocks the action with stderr feedback. JSON `{"continue": false, "stopReason": "..."}` also stops the teammate entirely, matching `Stop` hook behavior
PreToolUse	`hookSpecificOutput`	`permissionDecision` (allow/deny/ask/defer), `permissionDecisionReason`
PermissionRequest	`hookSpecificOutput`	`decision.behavior` (allow/deny)
PermissionDenied	`hookSpecificOutput`	`retry: true` tells the model it may retry the denied tool call
WorktreeCreate	path return	Command hook prints path on stdout; HTTP hook returns `hookSpecificOutput.worktreePath`. Hook failure or missing path fails creation
Elicitation	`hookSpecificOutput`	`action` (accept/decline/cancel), `content` (form field values for accept)
ElicitationResult	`hookSpecificOutput`	`action` (accept/decline/cancel), `content` (form field values override)
MessageDisplay	`hookSpecificOutput`	`displayContent` replaces the displayed text on screen. Display-only: the transcript and what Claude sees keep the original
SessionStart, Setup, SubagentStart	Context only	`hookSpecificOutput.additionalContext` adds context for Claude. SessionStart also accepts `initialUserMessage`, `watchPaths`, `sessionTitle`, and `reloadSkills`. No blocking or decision control
WorktreeRemove, Notification, SessionEnd, PostCompact, InstructionsLoaded, StopFailure, CwdChanged, FileChanged	None	No decision control. Used for side effects like logging or cleanup

A few events can also rewrite content rather than only allow or block it:

- `PreToolUse`: `updatedInput` directly under `hookSpecificOutput` replaces a tool's arguments

before it runs. See **PreToolUse decision control**

- `PermissionRequest`: `updatedInput` inside the `decision` object. See **PermissionRequest decision control**
- `PostToolUse`: `updatedToolOutput` replaces the tool's result. See **PostToolUse decision control**
- `UserPromptSubmit`: cannot replace the prompt; it only injects `additionalContext` alongside it

For redaction or transformation use cases, intercept at `PreToolUse` for outbound tool inputs and `PostToolUse` for inbound tool results.

Here are examples of each pattern in action:

Top-level decision

This example uses `${CLAUDE_PROJECT_DIR}` to run a style checker from the project's `.claude/hooks/` directory after any `Write` or `Edit` tool call:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/check-style.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

PreToolUse

Define plugin hooks in `hooks/hooks.json` with an optional top-level `description` field. When a

plugin is enabled, its hooks merge with your user and project hooks.This example runs a formatting script bundled with the plugin:

```
{
  "description": "Automatic code formatting",
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PLUGIN_ROOT}/scripts/format.sh",
            "args": [],
            "timeout": 30
          }
        ]
      }
    ]
  }
}
```

See the **plugin components reference** for details on creating plugin hooks.

PermissionRequest

Used by `UserPromptSubmit`, `UserPromptExpansion`, `PostToolUse`, `PostToolUseFailure`, `PostToolBatch`, `Stop`, `SubagentStop`, `ConfigChange`, and `PreCompact`. The only value is `"block"`. To allow the action to proceed, omit `decision` from your JSON, or exit 0 without any JSON at all:

```
{
  "decision": "block",
  "reason": "Test suite must pass before proceeding"
}
```

Uses `hookSpecificOutput` for richer control: allow, deny, or escalate to the user. You can also modify tool input before it runs or inject additional context for Claude. See **PreToolUse decision control** for the full set of options.

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Database writes are not allowed"
  }
}
```

Uses `hookSpecificOutput` to allow or deny a permission request on behalf of the user. When allowing, you can also modify the tool's input or apply permission rules so the user isn't prompted again. See **PermissionRequest decision control** for the full set of options.

```
{
  "hookSpecificOutput": {
    "hookEventName": "PermissionRequest",
    "decision": {
      "behavior": "allow",
      "updatedInput": {
        "command": "npm run lint"
      }
    }
  }
}
```

Register a command hook for the event in your settings file:

```
{
  "hooks": {
    "MessageDisplay": [
      {
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/plain-display.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

Save this script to `.claude/hooks/plain-display.sh` in your project and make it executable with `chmod +x`:

```
#!/bin/bash
jq '{hookSpecificOutput: {hookEventName: "MessageDisplay",
displayContent: (.delta | gsub("\\*\\*"; "") | gsub("`"; ""))}}'
```

The script needs `jq` on your `PATH`.

Register a command hook that runs the script through PowerShell:

```
{
  "hooks": {
    "MessageDisplay": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "powershell.exe",
            "args": [
              "-NoProfile",
              "-ExecutionPolicy",
              "Bypass",
              "-File",
              "${CLAUDE_PROJECT_DIR}/.claude/hooks/plain-
display.ps1"
            ]
          }
        ]
      }
    ]
  }
}
```

The `-NoProfile` flag skips loading your PowerShell profile so the hook starts fast, and `-ExecutionPolicy Bypass` lets PowerShell run the local script file.Save this script to `.claude/hooks/plain-display.ps1` in your project:

```
$batch = [Console]::In.ReadToEnd() | ConvertFrom-Json
$text = $batch.delta -replace '\*\*', '' -replace '``', ''
@{
  hookSpecificOutput = @{
    hookEventName = "MessageDisplay"
    displayContent = $text
  }
} | ConvertTo-Json
```

For extended examples including Bash command validation, prompt filtering, and auto-approval scripts, see **What you can automate** in the guide and the **Bash command validator reference implementation**.

Hook events

Each event corresponds to a point in Claude Code's lifecycle where hooks can run. The sections below are ordered to match the lifecycle: from session setup through the agentic loop to session end. Each section describes when the event fires, what matchers it supports, the JSON input it receives, and how to control behavior through output.

SessionStart

Runs when Claude Code starts a new session or resumes an existing session. Useful for loading development context like existing issues or recent changes to your codebase, or setting up environment variables. For static context that does not require a script, use **CLAUDE.md** instead.

SessionStart runs on every session, so keep these hooks fast. Only `type: "command"` and `type: "mcp_tool"` hooks are supported.

The matcher value corresponds to how the session was initiated:

Matcher	When it fires
`startup`	New session
`resume`	`--resume`, `--continue`, or `/resume`
`clear`	`/clear`
`compact`	Auto or manual compaction

SessionStart input

In addition to the **common input fields**, SessionStart hooks receive `source` and optionally `model`, `agent_type`, and `session_title`. The `source` field indicates how the session started: `"startup"` for new sessions, `"resume"` for resumed sessions, `"clear"` after `/clear`, or `"compact"` after compaction. The `model` field contains the active model identifier. It can be omitted, for example after `/clear` or when a session is restored through conversation recovery, so check for the field before reading it. If you start Claude Code with `claude --agent <name>`, an `agent_type` field contains the agent name. The `session_title` field carries the current session title if one is already set, for example via `--name` or `/rename`. A hook that emits `sessionTitle` can check `session_title` first to avoid overwriting a title the user set explicitly.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "SessionStart",
  "source": "startup",
  "model": "claude-sonnet-4-6"
}
```

SessionStart decision control

Any text your hook script prints to stdout is added as context for Claude. In addition to the **JSON output fields** available to all hooks, you can return these event-specific fields:

Field	Description
`additionalContext`	String added to Claude's context at the start of the conversation, before the first prompt. See **Add context for Claude** for how the text is delivered and what to put in it
`initialUserMessage`	String used as the first user message of the session. Applies in **non-interactive mode** (`-p`), where it becomes the first turn even if no prompt is provided. If a prompt is provided, it follows as the next turn. Unlike `additionalContext`, which attaches to an existing turn, this creates the turn
`sessionTitle`	Sets the session title, with the same effect as `/rename`. Use to name sessions automatically from the launch folder, git branch, or worktree name. Applies only when `source` is `"startup"` or `"resume"`; ignored on `"clear"` and `"compact"`
`watchPaths`	Array of absolute paths to watch for **FileChanged** events during this session
`reloadSkills`	Boolean. When `true`, Claude Code re-scans the **skill** and command directories after the SessionStart hooks complete, so skills the hook installed are available in the same session, starting with the first prompt

```
{
  "hookSpecificOutput": {
    "hookEventName": "SessionStart",
    "additionalContext": "Current branch: feat/auth-
refactor\nUncommitted changes: src/auth.ts, src/login.tsx\nActive
issue: #4211 Migrate to OAuth2",
    "sessionTitle": "auth-refactor"
  }
}
```

Since plain stdout already reaches Claude for this event, a hook that only loads context can print to stdout directly without building JSON. Use the JSON form when you need to combine context with other fields such as `suppressOutput` or `sessionTitle`.

Use `reloadSkills` when a SessionStart hook installs or updates skills. Skill discovery normally runs before SessionStart hooks finish, so files the hook writes into `~/.claude/skills/` or `.claude/skills/` would otherwise only appear in the next session. This example syncs a shared skills repository and requests the re-scan:

```
#!/bin/bash

git -C ~/.claude/skills/team-skills pull --quiet 2>/dev/null || \
  git clone --quiet https://git.example.com/your-org/team-
skills.git ~/.claude/skills/team-skills

echo '{"hookSpecificOutput": {"hookEventName": "SessionStart",
"reloadSkills": true}}'
```

Persist environment variables

SessionStart hooks have access to the `CLAUDE_ENV_FILE` environment variable, which provides a file path where you can persist environment variables for subsequent Bash commands.

To set individual environment variables, write `export` statements to `CLAUDE_ENV_FILE`. Use append (`>>`) to preserve variables set by other hooks:

```
#!/bin/bash

if [ -n "$CLAUDE_ENV_FILE" ]; then
  echo 'export NODE_ENV=production' >> "$CLAUDE_ENV_FILE"
  echo 'export DEBUG_LOG=true' >> "$CLAUDE_ENV_FILE"
  echo 'export PATH="$PATH:./node_modules/.bin"' >>
"$CLAUDE_ENV_FILE"
fi

exit 0
```

To capture all environment changes from setup commands, compare the exported variables before and after:

```
#!/bin/bash

ENV_BEFORE=$(export -p | sort)

# Run your setup commands that modify the environment
source ~/.nvm/nvm.sh
nvm use 20

if [ -n "$CLAUDE_ENV_FILE" ]; then
  ENV_AFTER=$(export -p | sort)
  comm -13 <(echo "$ENV_BEFORE") <(echo "$ENV_AFTER") >>
"$CLAUDE_ENV_FILE"
fi

exit 0
```

Any variables written to this file will be available in all subsequent Bash commands that Claude Code executes during the session.

> `CLAUDE_ENV_FILE` is available for SessionStart, **Setup**, **CwdChanged**, and **FileChanged** hooks. Other hook types do not have access to this variable.

Setup

Fires only when you launch Claude Code with `--init-only`, or with `--init` or `--maintenance` in print mode (`-p`). It does not fire on normal startup. Use it for one-time dependency installation or scheduled cleanup that you trigger explicitly from CI or scripts, separate from normal session startup. For per-session initialization, use **SessionStart** instead.

The matcher value corresponds to the CLI flag that triggered the hook:

Matcher	When it fires
`init`	`claude --init-only` or `claude -p --init`
`maintenance`	`claude -p --maintenance`

`--init-only` runs Setup hooks and `SessionStart` hooks with the `startup` matcher, then exits without starting a conversation. `--init` and `--maintenance` fire Setup hooks only when combined with `-p` (print mode); in an interactive session those two flags do not currently fire Setup hooks.

Because Setup does not fire on every launch, a plugin that needs a dependency installed cannot rely on Setup alone. The practical pattern is to check for the dependency on first use and install on miss, for example a hook or skill that tests for `${CLAUDE_PLUGIN_DATA}/node_modules` and runs `npm install` if absent. See the **persistent data directory** for where to store installed dependencies.

Setup input

In addition to the **common input fields**, Setup hooks receive a `trigger` field set to either `"init"` or `"maintenance"`:

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "Setup",
  "trigger": "init"
}
```

Setup decision control

Setup hooks cannot block. On exit code 2, stderr is shown to the user; on any other non-zero exit code, stderr appears only when you launch with `--verbose`. In both cases execution continues. To pass information into Claude's context, return `additionalContext` in JSON output; plain stdout is written to the debug log only. In addition to the **JSON output fields** available to all hooks, you can return these event-specific fields:

Field	Description
`additionalContext`	String added to Claude's context. Multiple hooks' values are concatenated

```
{
  "hookSpecificOutput": {
    "hookEventName": "Setup",
    "additionalContext": "Dependencies installed: node_modules,
.venv"
  }
}
```

Setup hooks have access to `CLAUDE_ENV_FILE`. Variables written to that file persist into subsequent Bash commands for the session, just as in **SessionStart hooks**. Only `type: "command"` and `type: "mcp_tool"` hooks are supported.

InstructionsLoaded

Fires when a `CLAUDE.md` or `.claude/rules/*.md` file is loaded into context. This event fires at session start for eagerly-loaded files and again later when files are lazily loaded, for example when Claude accesses a subdirectory that contains a nested `CLAUDE.md` or when conditional rules with `paths:` frontmatter match. The hook does not support blocking or decision control. It runs asynchronously for observability purposes.

The matcher runs against `load_reason`. For example, use `"matcher": "session_start"` to fire only for files loaded at session start, or `"matcher": "path_glob_match|nested_traversal"` to fire only for lazy loads.

InstructionsLoaded input

In addition to the **common input fields**, InstructionsLoaded hooks receive these fields:

Field	Description
`file_path`	Absolute path to the instruction file that was loaded
`memory_type`	Scope of the file: `"User"`, `"Project"`, `"Local"`, or `"Managed"`
`load_reason`	Why the file was loaded: `"session_start"`, `"nested_traversal"`, `"path_glob_match"`, `"include"`, or `"compact"`. The `"compact"` value fires when instruction files are re-loaded after a compaction event
`globs`	Path glob patterns from the file's `paths:` frontmatter, if any. Present only for `path_glob_match` loads
`trigger_file_path`	Path to the file whose access triggered this load, for lazy loads
`parent_file_path`	Path to the parent instruction file that included this one, for `include` loads

```
{
  "session_id": "abc123",
  "transcript_path":
"/Users/.../.claude/projects/.../transcript.jsonl",
  "cwd": "/Users/my-project",
  "hook_event_name": "InstructionsLoaded",
  "file_path": "/Users/my-project/CLAUDE.md",
  "memory_type": "Project",
  "load_reason": "session_start"
}
```


InstructionsLoaded decision control

InstructionsLoaded hooks have no decision control. They cannot block or modify instruction loading. Use this event for audit logging, compliance tracking, or observability.

UserPromptSubmit

Runs when the user submits a prompt, before Claude processes it. This allows you to add additional context based on the prompt/conversation, validate prompts, or block certain types of prompts.

`UserPromptSubmit` hooks have a default timeout of 30 seconds for `command`, `http`, and `mcp_tool` types, shorter than the 600-second default for those types on most other events. Because this hook runs before every prompt and blocks model processing until it completes, a stuck hook stalls the session. If your hook needs more time, set the `timeout` field in the hook entry.

UserPromptSubmit input

In addition to the **common input fields**, UserPromptSubmit hooks receive the `prompt` field containing the text the user submitted.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "UserPromptSubmit",
  "prompt": "Write a function to calculate the factorial of a
number"
}
```

UserPromptSubmit decision control

`UserPromptSubmit` hooks can control whether a user prompt is processed and add context. All **JSON output fields** are available.

There are two ways to add context to the conversation on exit code 0:

- **Plain text stdout**: any non-JSON text written to stdout is added as context
- **JSON with** `additionalContext` : use the JSON format below for more control. The `additionalContext` field is added as context

Plain stdout is shown as hook output in the transcript. The `additionalContext` field is added more discretely.

To block a prompt, return a JSON object with `decision` set to `"block"` :

Field	Description
`decision`	`"block"` prevents the prompt from being processed and erases it from context. Omit to allow the prompt to proceed
`reason`	Shown to the user when `decision` is `"block"`. Not added to context
`additionalContext`	String added to Claude's context alongside the submitted prompt. See **Add context for Claude**
`sessionTitle`	Sets the session title. Use to name sessions automatically based on the prompt content
`suppressOriginalPrompt`	If `true` when `decision` is `"block"`, omits the original prompt text from the block message shown to the user

```
{
  "decision": "block",
  "reason": "Explanation for decision",
  "hookSpecificOutput": {
    "hookEventName": "UserPromptSubmit",
    "additionalContext": "My additional context here",
    "sessionTitle": "My session title"
  }
}
```

The JSON format isn't required for simple use cases. To add context, you can print plain text to stdout with exit code 0. Use JSON when you need to block prompts or want more structured control.

UserPromptExpansion

Runs when a user-typed slash command expands into a prompt before reaching Claude. Use this to block specific commands from direct invocation, inject context for a particular skill, or log which commands users invoke. For example, a hook matching `deploy` can block `/deploy` unless an approval file is present, or a hook matching a review skill can append the team's review checklist as `additionalContext`.

This event covers the path `PreToolUse` does not: a `PreToolUse` hook matching the `Skill` tool fires only when Claude calls the tool, but typing `/skillname` directly bypasses `PreToolUse`. `UserPromptExpansion` fires on that direct path.

Matches on `command_name`. Leave the matcher empty to fire on every prompt-type slash command.

UserPromptExpansion input

In addition to the **common input fields**, UserPromptExpansion hooks receive `expansion_type`, `command_name`, `command_args`, `command_source`, and the original `prompt` string. The `expansion_type` field is `slash_command` for skill and custom commands, or `mcp_prompt` for MCP server prompts.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../00893aaf.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "UserPromptExpansion",
  "expansion_type": "slash_command",
  "command_name": "example-skill",
  "command_args": "arg1 arg2",
  "command_source": "plugin",
  "prompt": "/example-skill arg1 arg2"
}
```

UserPromptExpansion decision control

`UserPromptExpansion` hooks can block the expansion or add context. All **JSON output fields** are available.

Field	Description
`decision`	`"block"` prevents the slash command from expanding. Omit to allow it to proceed
`reason`	Shown to the user when `decision` is `"block"`
`additionalContext`	String added to Claude's context alongside the expanded prompt. See **Add context for Claude**

```
{
  "decision": "block",
  "reason": "This slash command is not available",
  "hookSpecificOutput": {
    "hookEventName": "UserPromptExpansion",
    "additionalContext": "Additional context for this expansion"
  }
}
```

MessageDisplay

Runs while an assistant message streams to the screen. Claude Code displays the message in increments: each time a batch of newly completed lines is ready to render, the hook runs once with those lines and Claude Code renders the hook's replacement text in their place. A long message

produces several calls; a short message may produce only one.

Use MessageDisplay to:

- strip markdown for a minimal display
- transform the text an Agent SDK application shows its users
- redact API keys or internal hostnames from Claude's responses

Claude Code holds each batch until your hook returns, so keep the hook fast. If the hook fails or times out, Claude Code displays the original text. The default timeout for this event is 10 seconds; if your hook needs more time, set the `timeout` field in the hook entry.

MessageDisplay is display-only: the replacement text changes only what is rendered on screen. The transcript and what Claude sees keep the original text, so Claude never sees the replacement, and verbose mode shows the original. The hook receives assistant message text only, so tool results and the text you type render unchanged.

MessageDisplay does not support matchers and fires for every assistant message that streams text; messages with no text, such as tool-call-only responses, do not trigger it.

In non-interactive runs, including Agent SDK queries and `claude -p`, MessageDisplay runs once per assistant message instead of once per batch of lines. The single call arrives after the message completes and carries the full message text: `index` is `0`, `final` is `true`, and `delta` holds the entire message. A hook that collects the `delta` text for each message receives the same total text in both modes.

MessageDisplay input

In addition to the **common input fields**, MessageDisplay hooks receive identifiers for the turn and message, the position of this call within the message, and the new text in `delta`. Batch boundaries depend on how the text streams, so use `index` and `final` to track progress through a message rather than expecting lines to be grouped a particular way.

Field	Description
`turn_id`	UUID of the current turn
`message_id`	UUID of the assistant message being displayed. Stable across every batch of the same message. This is not the API `msg_…` id, so it cannot be correlated with transcript message ids
`index`	Zero-based index of this batch within the message

Field	Description
`final`	`true` on the message's last batch. Each message has exactly one final batch
`delta`	The newly completed lines since the prior batch, terminating newlines included. Always whole lines, except the final batch which may end mid-line. In interactive runs, the final batch's delta is empty when the message ends on a newline, so treat `final`, not a non-empty delta, as the end-of-message signal. In Agent SDK and `claude -p` runs, the single call carries the entire message


```
{
  "session_id": "abc123",
  "transcript_path":
"/Users/.../.claude/projects/.../transcript.jsonl",
  "cwd": "/Users/my-project",
  "hook_event_name": "MessageDisplay",
  "turn_id": "0c9e6a2f-7d41-4f4e-9a15-3f4f7c2b8d10",
  "message_id": "5b2a9c8e-1f63-4d8a-b7c4-9e0d2a6f1c3b",
  "index": 0,
  "final": false,
  "delta": "Here is the plan:\n"
}
```


MessageDisplay output

In addition to the **JSON output fields** available to all hooks, MessageDisplay hooks can return `displayContent` to replace the delta on screen:

Field	Description
`displayContent`	Text displayed in place of the delta. Omit it to display the original

MessageDisplay hooks have no decision control. They cannot block the message or change what is stored in the transcript or sent to Claude.

This example strips markdown formatting from Claude's responses for a plain-text display. The script reads each batch from stdin, removes bold markers and inline code backticks from `delta`, and returns the result as `displayContent`.

macOS/Linux

This example uses `${CLAUDE_PROJECT_DIR}` to run a style checker from the project's

`.claude/hooks/` directory after any `Write` or `Edit` tool call:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/check-style.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

Windows (PowerShell)

Define plugin hooks in `hooks/hooks.json` with an optional top-level `description` field. When a plugin is enabled, its hooks merge with your user and project hooks.This example runs a formatting script bundled with the plugin:

```
{
  "description": "Automatic code formatting",
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PLUGIN_ROOT}/scripts/format.sh",
            "args": [],
            "timeout": 30
          }
        ]
      }
    ]
  }
}
```

See the **plugin components reference** for details on creating plugin hooks.

Used by `UserPromptSubmit`, `UserPromptExpansion`, `PostToolUse`, `PostToolUseFailure`, `PostToolBatch`, `Stop`, `SubagentStop`, `ConfigChange`, and `PreCompact`. The only value is `"block"`. To allow the action to proceed, omit `decision` from your JSON, or exit 0 without any JSON at all:

```
{
  "decision": "block",
  "reason": "Test suite must pass before proceeding"
}
```

Uses `hookSpecificOutput` for richer control: allow, deny, or escalate to the user. You can also modify tool input before it runs or inject additional context for Claude. See **PreToolUse decision control** for the full set of options.

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Database writes are not allowed"
  }
}
```

Uses `hookSpecificOutput` to allow or deny a permission request on behalf of the user. When allowing, you can also modify the tool's input or apply permission rules so the user isn't prompted again. See **PermissionRequest decision control** for the full set of options.

```
{
  "hookSpecificOutput": {
    "hookEventName": "PermissionRequest",
    "decision": {
      "behavior": "allow",
      "updatedInput": {
        "command": "npm run lint"
      }
    }
  }
}
```

Register a command hook for the event in your settings file:

```
{
  "hooks": {
    "MessageDisplay": [
      {
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/plain-display.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

Save this script to `.claude/hooks/plain-display.sh` in your project and make it executable with `chmod +x`:

```
#!/bin/bash
jq '{hookSpecificOutput: {hookEventName: "MessageDisplay",
displayContent: (.delta | gsub("\\*\\*"; "") | gsub("`"; ""))}}'
```

The script needs `jq` on your `PATH`.

Register a command hook that runs the script through PowerShell:

```
{
  "hooks": {
    "MessageDisplay": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "powershell.exe",
            "args": [
              "-NoProfile",
              "-ExecutionPolicy",
              "Bypass",
              "-File",
              "${CLAUDE_PROJECT_DIR}/.claude/hooks/plain-
display.ps1"
            ]
          }
        ]
      }
    ]
  }
}
```

The `-NoProfile` flag skips loading your PowerShell profile so the hook starts fast, and `-ExecutionPolicy Bypass` lets PowerShell run the local script file.Save this script to `.claude/hooks/plain-display.ps1` in your project:

```
$batch = [Console]::In.ReadToEnd() | ConvertFrom-Json
$text = $batch.delta -replace '\*\*', '' -replace '``', ''
@{
  hookSpecificOutput = @{
    hookEventName = "MessageDisplay"
    displayContent = $text
  }
} | ConvertTo-Json
```

Batches with no markdown pass through unchanged. If the script fails, for example because `jq` is missing, Claude Code displays the original text and notes the failure only in **debug output**, not in the session.

PreToolUse

Runs after Claude creates tool parameters and before processing the tool call. Matches on tool name: `Bash`, `Edit`, `Write`, `Read`, `Glob`, `Grep`, `Agent`, `WebFetch`, `WebSearch`, `AskUserQuestion`, `ExitPlanMode`, and any **MCP tool names**.

Use **PreToolUse decision control** to allow, deny, ask, or defer the tool call.

PreToolUse input

In addition to the **common input fields**, PreToolUse hooks receive `tool_name`, `tool_input`, and `tool_use_id`. The `tool_input` fields depend on the tool:

Bash

Executes shell commands.

Field	Type	Example	Description
`command`	string	`"npm test"`	The shell command to execute
`description`	string	`"Run test suite"`	Optional description of what the command does
`timeout`	number	`120000`	Optional timeout in milliseconds
`run_in_background`	boolean	`false`	Whether to run the command in background

Write

Creates or overwrites a file.

Field	Type	Example	Description
`file_path`	string	`"/path/to/file.txt"`	Absolute path to the file to write
`content`	string	`"file content"`	Content to write to the file

Edit

Replaces a string in an existing file.

Field	Type	Example	Description
`file_path`	string	`"/path/to/file.txt"`	Absolute path to the file to edit

Field	Type	Example	Description
`old_string`	string	`"original text"`	Text to find and replace
`new_string`	string	`"replacement text"`	Replacement text
`replace_all`	boolean	`false`	Whether to replace all occurrences

Read

Reads file contents.

Field	Type	Example	Description
`file_path`	string	`"/path/to/file.txt"`	Absolute path to the file to read
`offset`	number	`10`	Optional line number to start reading from
`limit`	number	`50`	Optional number of lines to read

Glob

Finds files matching a glob pattern.

Field	Type	Example	Description
`pattern`	string	`"**/*.ts"`	Glob pattern to match files against
`path`	string	`"/path/to/dir"`	Optional directory to search in. Defaults to current working directory

Grep

Searches file contents with regular expressions.

Field	Type	Example	Description
`pattern`	string	`"TODO.*fix"`	Regular expression pattern to search for
`path`	string	`"/path/to/dir"`	Optional file or directory to search in
`glob`	string	`"*.ts"`	Optional glob pattern to filter files
`output_mode`	string	`"content"`	`"content"`, `"files_with_matches"`, or `"count"`. Defaults to `"files_with_matches"`

Field	Type	Example	Description
`-i`	boolean	`true`	Case insensitive search
`multiline`	boolean	`false`	Enable multiline matching

WebFetch

Fetches and processes web content.

Field	Type	Example	Description
`url`	string	`"https://example.com/api"`	URL to fetch content from
`prompt`	string	`"Extract the API endpoints"`	Prompt to run on the fetched content

WebSearch

Searches the web.

Field	Type	Example	Description
`query`	string	`"react hooks best practices"`	Search query
`allowed_domains`	array	`["docs.example.com"]`	Optional: only include results from these domains
`blocked_domains`	array	`["spam.example.com"]`	Optional: exclude results from these domains

Agent

Spawns a **subagent**.

Field	Type	Example	Description
`prompt`	string	`"Find all API endpoints"`	The task for the agent to perform
`description`	string	`"Find API endpoints"`	Short description of the task
`subagent_type`	string	`"Explore"`	Type of specialized agent to use
`model`	string	`"sonnet"`	Optional model alias to override the default

In `PostToolUse`, `tool_response` for a completed Agent call carries the subagent's final text along with usage telemetry. Read these fields to record per-subagent cost from a hook:

Field	Type	Example	Description
`status`	string	`"completed"`	`"completed"` for synchronous calls, `"async_launched"` for `run_in_background: true`
`agentId`	string	`"a4d2c8f1e0b3a297"`	Identifier for the subagent run
`content`	array	`[{"type": "text", "text": "Found 12 endpoints..."}]`	The subagent's final text blocks
`resolvedModel`	string	`"claude-sonnet-4-5"`	Model the subagent ran on, which may differ from the requested model. Requires Claude Code v2.1.174 or later
`totalTokens`	number	`12450`	Total tokens billed across the subagent's turns
`totalDurationMs`	number	`48211`	Wall-clock duration of the subagent run
`totalToolUseCount`	number	`7`	Count of tool calls the subagent made
`usage`	object	`{"input_tokens": 8320, ...}`	Per-type token breakdown: `input_tokens`, `output_tokens`, `cache_creation_input_tokens`, `cache_read_input_tokens`

For `run_in_background: true` calls, the tool returns immediately after launching the subagent, so `tool_response` carries no usage fields. It has `status: "async_launched"`, `agentId`, `description`, `prompt`, `outputFile`, and `resolvedModel`.

The `resolvedModel` field names the model the subagent actually runs on, which can differ from the

`model` value in `tool_input`, such as when `availableModels` or another override applies. It requires Claude Code v2.1.174 or later.

AskUserQuestion

Asks the user one to four multiple-choice questions.

Field	Type	Example	Description
`questions`	array	`[{"question": "Which framework?", "header": "Framework", "options": [{"label": "React"}], "multiSelect": false}]`	Questions to present, each with a `question` string, short `header`, `options` array, and optional `multiSelect` flag
`answers`	object	`{"Which framework?": "React"}`	Optional. Maps question text to the selected option label. Multi-select answers join labels with commas. Claude does not set this field; supply it via `updatedInput` to answer programmatically

ExitPlanMode

Presents a plan and asks the user to approve it before Claude leaves **plan mode**. Claude writes the plan to a file on disk before calling the tool, so the literal `tool_input` from the model only carries `allowedPrompts`. Claude Code injects the plan content and file path before passing the input to hooks.

Field	Type	Example	Description
`plan`	string	`"## Refactor auth\n1. Extract..."`	Plan content in Markdown. Injected from the plan file on disk
`planFilePath`	string	`"/Users/.../plans/refactor-auth.md"`	Path to the plan file. Injected
`allowedPrompts`	array	`[{"tool": "Bash", "prompt": "run tests"}]`	Optional. Prompt-based permissions Claude is requesting to implement the plan, each with a `tool` name and a `prompt` describing the category of action

In `PostToolUse`, `tool_response` is an object with `plan` and `filePath` fields holding the approved plan, plus internal status flags. Read `tool_response.plan` for the plan content rather than re-reading the file from disk.

PreToolUse decision control

`PreToolUse` hooks can control whether a tool call proceeds. Unlike other hooks that use a top-level `decision` field, PreToolUse returns its decision inside a `hookSpecificOutput` object. This gives it richer control: four outcomes (allow, deny, ask, or defer) plus the ability to modify tool input before execution.

Field	Description
`permissionDecision`	`"allow"` skips the permission prompt. `"deny"` prevents the tool call. `"ask"` prompts the user to confirm. `"defer"` exits gracefully so the tool can be resumed later. **Deny and ask rules** are still evaluated regardless of what the hook returns
`permissionDecisionReason`	For `"allow"` and `"ask"`, shown to the user but not Claude. For `"deny"`, shown to Claude. For `"defer"`, ignored
`updatedInput`	Modifies the tool's input parameters before execution. Replaces the entire input object, so include unchanged fields alongside modified ones. Combine with `"allow"` to auto-approve, or `"ask"` to show the modified input to the user. For `"defer"`, ignored
`additionalContext`	String added to Claude's context alongside the tool result. Ignored when `permissionDecision` is `"defer"`. See **Add context for Claude**

When multiple PreToolUse hooks return different decisions, precedence is `deny` > `defer` > `ask` > `allow`.

When a hook returns `"ask"`, the permission prompt displayed to the user includes a label identifying where the hook came from: for example, `[User]`, `[Project]`, `[Plugin]`, or `[Local]`. This helps users understand which configuration source is requesting confirmation.

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "allow",
    "permissionDecisionReason": "My reason here",
    "updatedInput": {
      "field_to_modify": "new value"
    },
    "additionalContext": "Current environment: production.
Proceed with caution."
  }
}
```

`AskUserQuestion` and `ExitPlanMode` require user interaction and normally block in **non-**

interactive mode with the `-p` flag. Returning `permissionDecision: "allow"` together with `updatedInput` satisfies that requirement: the hook reads the tool's input from stdin, collects the answer through your own UI, and returns it in `updatedInput` so the tool runs without prompting. Returning `"allow"` alone is not sufficient for these tools. For `AskUserQuestion`, echo back the original `questions` array and add an `answers` object mapping each question's text to the chosen answer.

> PreToolUse previously used top-level `decision` and `reason` fields, but these are deprecated for this event. Use `hookSpecificOutput.permissionDecision` and `hookSpecificOutput.permissionDecisionReason` instead. The deprecated values `"approve"` and `"block"` map to `"allow"` and `"deny"` respectively. Other events like PostToolUse and Stop continue to use top-level `decision` and `reason` as their current format.

Defer a tool call for later

`"defer"` is for integrations that run `claude -p` as a subprocess and read its JSON output, such as an Agent SDK app or a custom UI built on top of Claude Code. It lets that calling process pause Claude at a tool call, collect input through its own interface, and resume where it left off. Claude Code honors this value only in **non-interactive mode** with the `-p` flag. In interactive sessions it logs a warning and ignores the hook result.

> The `defer` value requires Claude Code v2.1.89 or later. Earlier versions do not recognize it and the tool proceeds through the normal permission flow.

The `AskUserQuestion` tool is the typical case: Claude wants to ask the user something, but there is no terminal to answer in. The round trip works like this:

1. Claude calls `AskUserQuestion`. The `PreToolUse` hook fires.
2. The hook returns `permissionDecision: "defer"`. The tool does not execute. The process exits with `stop_reason: "tool_deferred"` and the pending tool call preserved in the transcript.
3. The calling process reads `deferred_tool_use` from the SDK result, surfaces the question in its own UI, and waits for an answer.
4. The calling process runs `claude -p --resume <session-id>`. The same tool call fires `PreToolUse` again.
5. The hook returns `permissionDecision: "allow"` with the answer in `updatedInput`. The tool executes and Claude continues.

The `deferred_tool_use` field carries the tool's `id`, `name`, and `input`. The `input` is the parameters Claude generated for the tool call, captured before execution:

```
{
  "type": "result",
  "subtype": "success",
  "stop_reason": "tool_deferred",
  "session_id": "abc123",
  "deferred_tool_use": {
    "id": "toolu_01abc",
    "name": "AskUserQuestion",
    "input": { "questions": [{ "question": "Which framework?",
"header": "Framework", "options": [{"label": "React"}, {"label":
"Vue"}], "multiSelect": false }] }
  }
}
```

There is no timeout or retry limit. The session remains on disk until you resume it, subject to the `cleanupPeriodDays` retention sweep that deletes session files after 30 days by default. If the answer is not ready when you resume, the hook can return `"defer"` again and the process exits the same way. The calling process controls when to break the loop by eventually returning `"allow"` or `"deny"` from the hook.

`"defer"` only works when Claude makes a single tool call in the turn. If Claude makes several tool calls at once, `"defer"` is ignored with a warning and the tool proceeds through the normal permission flow. The constraint exists because resume can only re-run one tool: there is no way to defer one call from a batch without leaving the others unresolved.

If the deferred tool is no longer available when you resume, the process exits with `stop_reason: "tool_deferred_unavailable"` and `is_error: true` before the hook fires. This happens when an MCP server that provided the tool is not connected for the resumed session. The `deferred_tool_use` payload is still included so you can identify which tool went missing.

> `--resume` restores the permission mode that was active when the tool was deferred, so you do not need to pass `--permission-mode` again. The exceptions are `plan` and `bypassPermissions`, which are never carried over. Passing `--permission-mode` explicitly on resume overrides the restored value.

PermissionRequest

Runs when the user is shown a permission dialog. Use **PermissionRequest decision control** to allow or deny on behalf of the user.

Matches on tool name, same values as PreToolUse.

PermissionRequest input

PermissionRequest hooks receive `tool_name` and `tool_input` fields like PreToolUse hooks, but without `tool_use_id`. An optional `permission_suggestions` array contains the "always allow" options the user would normally see in the permission dialog. The difference is when the hook fires: PermissionRequest hooks run when a permission dialog is about to be shown to the user, while PreToolUse hooks run before tool execution regardless of permission status.


```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "PermissionRequest",
  "tool_name": "Bash",
  "tool_input": {
    "command": "rm -rf node_modules",
    "description": "Remove node_modules directory"
  },
  "permission_suggestions": [
    {
      "type": "addRules",
      "rules": [{ "toolName": "Bash", "ruleContent": "rm -rf
node_modules" }],
      "behavior": "allow",
      "destination": "localSettings"
    }
  ]
}
```


PermissionRequest decision control

`PermissionRequest` hooks can allow or deny permission requests. In addition to the **JSON output fields** available to all hooks, your hook script can return a `decision` object with these event-specific fields:

Field	Description
`behavior`	`"allow"` grants the permission, `"deny"` denies it. **Deny and ask rules** are still evaluated, so a hook returning `"allow"` does not override a matching deny rule
`updatedInput`	For `"allow"` only: modifies the tool's input parameters before execution. Replaces the entire input object, so include unchanged fields alongside modified ones. The modified

Field	Description
	input is re-evaluated against deny and ask rules
`updatedPermissions`	For `"allow"` only: array of **permission update entries** to apply, such as adding an allow rule or changing the session permission mode
`message`	For `"deny"` only: tells Claude why the permission was denied
`interrupt`	For `"deny"` only: if `true`, stops Claude

```
{
  "hookSpecificOutput": {
    "hookEventName": "PermissionRequest",
    "decision": {
      "behavior": "allow",
      "updatedInput": {
        "command": "npm run lint"
      }
    }
  }
}
```

Permission update entries

The `updatedPermissions` output field and the `permission_suggestions` **input field** both use the same array of entry objects. Each entry has a `type` that determines its other fields, and a `destination` that controls where the change is written.

`type`	Fields	Effect
`addRules`	`rules`, `behavior`, `destination`	Adds permission rules. `rules` is an array of `{toolName, ruleContent?}` objects. Omit `ruleContent` to match the whole tool. `behavior` is `"allow"`, `"deny"`, or `"ask"`
`replaceRules`	`rules`, `behavior`, `destination`	Replaces all rules of the given `behavior` at the `destination` with the provided `rules`
`removeRules`	`rules`, `behavior`, `destination`	Removes matching rules of the given `behavior`
`setMode`	`mode`, `destination`	Changes the permission mode. Valid modes are `default`, `auto`, `acceptEdits`, `dontAsk`, `bypassPermissions`, and `plan`
`addDirectories`	`directories`, `destination`	Adds working directories. `directories` is an array of path strings

type	Fields	Effect
removeDirectories	directories, destination	Removes working directories

setMode with bypassPermissions only takes effect if the session was launched with bypass mode already available: --dangerously-skip-permissions, --permission-mode bypassPermissions, --allow-dangerously-skip-permissions, or permissions.defaultMode: "bypassPermissions" in settings, and the mode is not disabled by permissions.disableBypassPermissionsMode. Otherwise the update is a no-op. bypassPermissions is never persisted as defaultMode regardless of destination.

The destination field on every entry determines whether the change stays in memory or persists to a settings file.

destination	Writes to
session	in-memory only, discarded when the session ends
localSettings	.claude/settings.local.json
projectSettings	.claude/settings.json
userSettings	~/.claude/settings.json

A hook can echo one of the permission_suggestions it received as its own updatedPermissions output, which is equivalent to the user selecting that "always allow" option in the dialog.

PostToolUse

Runs immediately after a tool completes successfully.

Matches on tool name, same values as PreToolUse.

PostToolUse input

PostToolUse hooks fire after a tool has already executed successfully. The input includes both tool_input, the arguments sent to the tool, and tool_response, the result it returned. The exact schema for both depends on the tool.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "PostToolUse",
  "tool_name": "Write",
  "tool_input": {
    "file_path": "/path/to/file.txt",
    "content": "file content"
  },
  "tool_response": {
    "filePath": "/path/to/file.txt",
    "success": true
  },
  "tool_use_id": "toolu_01ABC123...",
  "duration_ms": 12
}
```

Field	Description
`duration_ms`	Optional. Tool execution time in milliseconds. Excludes time spent in permission prompts and PreToolUse hooks

PostToolUse decision control

`PostToolUse` hooks can provide feedback to Claude after tool execution. In addition to the **JSON output fields** available to all hooks, your hook script can return these event-specific fields:

Field	Description
`decision`	`"block"` adds the `reason` next to the tool result. Claude still sees the original output; to replace it, use `updatedToolOutput`
`reason`	Explanation shown to Claude when `decision` is `"block"`
`additionalContext`	String added to Claude's context alongside the tool result. See **Add context for Claude**
`updatedToolOutput`	Replaces the tool's output with the provided value before it is sent to Claude. The value must match the tool's output shape
`updatedMCPToolOutput`	Replaces the output for **MCP tools** only. Prefer `updatedToolOutput`, which works for all tools

The example below replaces the output of a `Bash` call. The replacement value matches the `Bash` tool's output shape:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PostToolUse",
    "additionalContext": "Additional information for Claude",
    "updatedToolOutput": {
      "stdout": "[redacted]",
      "stderr": "",
      "interrupted": false,
      "isImage": false
    }
  }
}
```

⚠ `updatedToolOutput` only changes what Claude sees. The tool has already run by the time the hook fires, so any files written, commands executed, or network requests sent have already taken effect. Telemetry such as OpenTelemetry tool spans and analytics events also captures the original output before the hook runs. To prevent or modify a tool call before it runs, use a **PreToolUse** hook instead.

The replacement value must match the tool's output shape. Built-in tools return structured objects rather than plain strings. For example, `Bash` returns an object with `stdout`, `stderr`, `interrupted`, and `isImage` fields. For built-in tools, a value that does not match the tool's output schema is ignored and the original output is used. MCP tool output is passed through without schema validation. Stripping error details that Claude needs can cause it to proceed on a false assumption.

PostToolUseFailure

Runs when a tool execution fails. This event fires for tool calls that throw errors or return failure results. Use this to log failures, send alerts, or provide corrective feedback to Claude.

Matches on tool name, same values as PreToolUse.

PostToolUseFailure input

PostToolUseFailure hooks receive the same `tool_name` and `tool_input` fields as PostToolUse, along with error information as top-level fields:

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "PostToolUseFailure",
  "tool_name": "Bash",
  "tool_input": {
    "command": "npm test",
    "description": "Run test suite"
  },
  "tool_use_id": "toolu_01ABC123...",
  "error": "Command exited with non-zero status code 1",
  "is_interrupt": false,
  "duration_ms": 4187
}
```

Field	Description
`error`	String describing what went wrong
`is_interrupt`	Optional boolean indicating whether the failure was caused by user interruption
`duration_ms`	Optional. Tool execution time in milliseconds. Excludes time spent in permission prompts and PreToolUse hooks

PostToolUseFailure decision control

`PostToolUseFailure` hooks can provide context to Claude after a tool failure. In addition to the **JSON output fields** available to all hooks, your hook script can return these event-specific fields:

Field	Description
`additionalContext`	String added to Claude's context alongside the error. See **Add context for Claude**

```
{
  "hookSpecificOutput": {
    "hookEventName": "PostToolUseFailure",
    "additionalContext": "Additional information about the
failure for Claude"
  }
}
```

PostToolBatch

Runs once after every tool call in a batch has resolved, before Claude Code sends the next request to the model. `PostToolUse` fires once per tool, which means it fires concurrently when Claude makes parallel tool calls. `PostToolBatch` fires exactly once with the full batch, so it is the right place to inject context that depends on the set of tools that ran rather than on any single tool. There is no matcher for this event.

PostToolBatch input

In addition to the **common input fields**, PostToolBatch hooks receive `tool_calls`, an array describing every tool call in the batch:

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "PostToolBatch",
  "tool_calls": [
    {
      "tool_name": "Read",
      "tool_input": {"file_path": "/.../ledger/accounts.py"},
      "tool_use_id": "toolu_01...",
      "tool_response": "     1\tfrom __future__ import
annotations\n     2\t..."
    },
    {
      "tool_name": "Read",
      "tool_input": {"file_path": "/.../ledger/transactions.py"},
      "tool_use_id": "toolu_02...",
      "tool_response": "     1\tfrom __future__ import
annotations\n     2\t..."
    }
  ]
}
```


`tool_response` contains the same content the model receives in the corresponding `tool_result` block. The value is a serialized string or content-block array, exactly as the tool emitted it. For `Read`, that means line-number-prefixed text rather than raw file contents. Responses can be large, so parse only the fields you need.

> The `tool_response` shape differs from `PostToolUse`'s. `PostToolUse` passes the tool's structured `Output` object, such as `{filePath: "...", success: true}` for `Write`; `PostToolBatch` passes the serialized `tool_result` content the model sees.

PostToolBatch decision control

`PostToolBatch` hooks can inject context for Claude. In addition to the **JSON output fields** available to all hooks, your hook script can return these event-specific fields:

Field	Description
`additionalContext`	Context string injected once before the next model call. See **Add context for Claude** for delivery details, what to put in it, and how resumed sessions handle past values

```
{
  "hookSpecificOutput": {
    "hookEventName": "PostToolBatch",
    "additionalContext": "These files are part of the ledger
module. Run pytest before marking the task complete."
  }
}
```

Returning `decision: "block"` or `continue: false` stops the agentic loop before the next model call.

PermissionDenied

Runs when the **auto mode** classifier denies a tool call. This hook only fires in auto mode: it does not run when you manually deny a permission dialog, when a `PreToolUse` hook blocks a call, or when a `deny` rule matches. Use it to log classifier denials, adjust configuration, or tell the model it may retry the tool call.

Matches on tool name, same values as PreToolUse.

PermissionDenied input

In addition to the **common input fields**, PermissionDenied hooks receive `tool_name`, `tool_input`, `tool_use_id`, and `reason`.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "auto",
  "hook_event_name": "PermissionDenied",
  "tool_name": "Bash",
  "tool_input": {
    "command": "rm -rf /tmp/build",
    "description": "Clean build directory"
  },
  "tool_use_id": "toolu_01ABC123...",
  "reason": "Auto mode denied: command targets a path outside the
project"
}
```

Field	Description
`reason`	The classifier's explanation for why the tool call was denied

PermissionDenied decision control

PermissionDenied hooks can tell the model it may retry the denied tool call. Return a JSON object with `hookSpecificOutput.retry` set to `true`:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PermissionDenied",
    "retry": true
  }
}
```

When `retry` is `true`, Claude Code adds a message to the conversation telling the model it may retry the tool call. The denial itself is not reversed. If your hook does not return JSON, or returns `retry: false`, the denial stands and the model receives the original rejection message.

Notification

Runs when Claude Code sends notifications. Matches on notification type: `permission_prompt`, `idle_prompt`, `auth_success`, `elicitation_dialog`, `elicitation_complete`, `elicitation_response`. Omit the matcher to run hooks for all notification types.

Use separate matchers to run different handlers depending on the notification type. This configuration triggers a permission-specific alert script when Claude needs permission approval and a different notification when Claude has been idle:

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "permission_prompt",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/permission-alert.sh"
          }
        ]
      },
      {
        "matcher": "idle_prompt",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/idle-notification.sh"
          }
        ]
      }
    ]
  }
}
```

Notification input

In addition to the **common input fields**, Notification hooks receive `message` with the notification text, an optional `title`, and `notification_type` indicating which type fired.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "Notification",
  "message": "Claude needs your permission",
  "title": "Permission needed",
  "notification_type": "permission_prompt"
}
```

Notification hooks cannot block or modify notifications. They are intended for side effects such as forwarding the notification to an external service. The **common JSON output fields** such as `systemMessage` apply.

SubagentStart

Runs when a Claude Code subagent is spawned via the Agent tool. Supports matchers to filter by agent type name. For built-in agents, this is the agent name like `general-purpose`, `Explore`, or `Plan`. For **custom subagents**, this is the `name` field from the agent's frontmatter, not the filename.

SubagentStart input

In addition to the **common input fields**, SubagentStart hooks receive `agent_id` with the unique identifier for the subagent and `agent_type` with the agent name (built-in agents like `"general-purpose"`, `"Explore"`, `"Plan"`, or custom agent names).

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "SubagentStart",
  "agent_id": "agent-abc123",
  "agent_type": "Explore"
}
```

SubagentStart hooks cannot block subagent creation, but they can inject context into the subagent. In addition to the **JSON output fields** available to all hooks, you can return:

Field	Description
`additionalContext`	String added to the subagent's context at the start of its conversation, before its first prompt. See **Add context for Claude**

```
{
  "hookSpecificOutput": {
    "hookEventName": "SubagentStart",
    "additionalContext": "Follow security guidelines for this
task"
  }
}
```

SubagentStop

Runs when a Claude Code subagent has finished responding. Matches on agent type, same values as SubagentStart.

SubagentStop input

In addition to the **common input fields**, SubagentStop hooks receive `stop_hook_active`, `agent_id`, `agent_type`, `agent_transcript_path`, and `last_assistant_message`. The `agent_type` field is the value used for matcher filtering. The `transcript_path` is the main session's transcript, while `agent_transcript_path` is the subagent's own transcript stored in a nested `subagents/` folder. The `last_assistant_message` field contains the text content of the subagent's final response, so hooks can access it without parsing the transcript file.

SubagentStop hooks also receive the `background_tasks` and `session_crons` arrays described under **Stop input**, available in Claude Code v2.1.145 or later. Both arrays are scoped to the parent session, not the subagent.

```
{
  "session_id": "abc123",
  "transcript_path": "~/.claude/projects/.../abc123.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "SubagentStop",
  "stop_hook_active": false,
  "agent_id": "def456",
  "agent_type": "Explore",
  "agent_transcript_path":
"~/.claude/projects/.../abc123/subagents/agent-def456.jsonl",
  "last_assistant_message": "Analysis complete. Found 3 potential
issues...",
  "background_tasks": [],
  "session_crons": []
}
```


SubagentStop hooks use the same decision control format as **Stop hooks**, including `hookSpecificOutput.additionalContext` with `hookEventName` set to `"SubagentStop"`, for non-error feedback that keeps the subagent running. Returning `decision: "block"` with a `reason` keeps the subagent running and delivers `reason` to the subagent as its next instruction. To inject context into the parent session after a subagent returns, use a `PostToolUse` hook on the `Agent` tool instead.

TaskCreated

Runs when a task is being created via the `TaskCreate` tool. Use this to enforce naming conventions, require task descriptions, or prevent certain tasks from being created.

When a `TaskCreated` hook exits with code 2, the task is not created and the stderr message is fed back to the model as feedback. To stop the teammate entirely instead of re-running it, return JSON with `{"continue": false, "stopReason": "..."}`. TaskCreated hooks do not support matchers and fire on every occurrence.

TaskCreated input

In addition to the **common input fields**, TaskCreated hooks receive `task_id`, `task_subject`, and optionally `task_description`, `teammate_name`, and `team_name`.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "TaskCreated",
  "task_id": "task-001",
  "task_subject": "Implement user authentication",
  "task_description": "Add login and signup endpoints",
  "teammate_name": "implementer",
  "team_name": "session-a1b2c3d4"
}
```

Field	Description
`task_id`	Identifier of the task being created
`task_subject`	Title of the task
`task_description`	Detailed description of the task. May be absent
`teammate_name`	Name of the teammate creating the task. May be absent
`team_name`	Deprecated. Session-derived team name; will be removed in a future release

TaskCreated decision control

TaskCreated hooks support two ways to control task creation:

- **Exit code 2**: the task is not created and the stderr message is fed back to the model as feedback.
- **JSON** `{"continue": false, "stopReason": "..."}` : stops the teammate entirely, matching `Stop` hook behavior. The `stopReason` is shown to the user.

This example blocks tasks whose subjects don't follow the required format:

```
#!/bin/bash
INPUT=$(cat)
TASK_SUBJECT=$(echo "$INPUT" | jq -r '.task_subject')

if [[ ! "$TASK_SUBJECT" =~ ^\[TICKET-[0-9]+\] ]]; then
  echo "Task subject must start with a ticket number, e.g.
'[TICKET-123] Add feature'" >&2
  exit 2
fi

exit 0
```

TaskCompleted

Runs when a task is being marked as completed. This fires in two situations: when any agent explicitly marks a task as completed through the TaskUpdate tool, or when an **agent team** teammate finishes its turn with in-progress tasks. Use this to enforce completion criteria like passing tests or lint checks before a task can close.

When a `TaskCompleted` hook exits with code 2, the task is not marked as completed and the stderr message is fed back to the model as feedback. To stop the teammate entirely instead of re-running it, return JSON with `{"continue": false, "stopReason": "..."}`. TaskCompleted hooks do not support matchers and fire on every occurrence.

TaskCompleted input

In addition to the **common input fields**, TaskCompleted hooks receive `task_id`, `task_subject`, and optionally `task_description`, `teammate_name`, and `team_name`.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "TaskCompleted",
  "task_id": "task-001",
  "task_subject": "Implement user authentication",
  "task_description": "Add login and signup endpoints",
  "teammate_name": "implementer",
  "team_name": "session-a1b2c3d4"
}
```

Field	Description
`task_id`	Identifier of the task being completed
`task_subject`	Title of the task
`task_description`	Detailed description of the task. May be absent
`teammate_name`	Name of the teammate completing the task. May be absent
`team_name`	Deprecated. Session-derived team name; will be removed in a future release

TaskCompleted decision control

TaskCompleted hooks support two ways to control task completion:

- **Exit code 2**: the task is not marked as completed and the stderr message is fed back to the model as feedback.
- **JSON** `{"continue": false, "stopReason": "..."}`: stops the teammate entirely, matching `Stop` hook behavior. The `stopReason` is shown to the user.

This example runs tests and blocks task completion if they fail:

```
#!/bin/bash
INPUT=$(cat)
TASK_SUBJECT=$(echo "$INPUT" | jq -r '.task_subject')

# Run the test suite
if ! npm test 2>&1; then
  echo "Tests not passing. Fix failing tests before completing:
$TASK_SUBJECT" >&2
  exit 2
fi

exit 0
```

Stop

Runs when the main Claude Code agent has finished responding. Does not run if the stoppage occurred due to a user interrupt. API errors fire **StopFailure** instead.

> The `/goal` command is a built-in shortcut for a session-scoped prompt-based Stop hook. Use it when you want Claude to keep working until a condition holds without writing hook configuration.

Stop input

In addition to the **common input fields**, Stop hooks receive `stop_hook_active`, `last_assistant_message`, `background_tasks`, and `session_crons`. The `stop_hook_active` field is `true` when Claude Code is already continuing as a result of a stop hook. Check this value or process the transcript to avoid blocking on a condition that will never resolve. Claude Code overrides the hook and ends the turn after 8 consecutive blocks.

The `last_assistant_message` field contains the text content of Claude's final response, so hooks can access it without parsing the transcript file.

The `background_tasks` and `session_crons` arrays, available in Claude Code v2.1.145 or later, let hooks distinguish "session is done" from "session is paused waiting for background work to wake it back up". Both arrays are present when the task registry is reachable and are empty when nothing is in flight or scheduled.

Each entry in `background_tasks` describes one in-flight task and uses these fields:

Field	Description
`id`	Task identifier
`type`	Friendly task-type label such as `shell`, `subagent`, `monitor`, `workflow`, `teammate`, `cloud session`, or `MCP task`. Each label identifies which Claude Code feature created the task. Falls back to the raw discriminant for unrecognized types
`status`	Current task status
`description`	Free-text description, capped at 1000 characters with an in-string `… [+N chars]` marker when clipped
`command`	Shell command line, capped at 1000 characters. Present only for `shell` tasks
`agent_type`	Subagent type name. Present only for `subagent` tasks
`server`	MCP server name. Present only for `monitor` and `MCP task` tasks
`tool`	MCP tool name. Present only for `monitor` and `MCP task` tasks
`name`	Workflow name. Present only for `workflow` tasks

Each entry in `session_crons` describes one session-scoped scheduled wakeup, sourced from `CronCreate`, `ScheduleWakeup`, and `/loop`:

Field	Description
`id`	Cron task identifier
`schedule`	Cron expression, for example `0 9 * * 1-5`
`recurring`	`false` for one-shot wakeups whose schedule encodes a single fire time, `true` for tasks that re-fire on every match
`prompt`	Prompt submitted when the cron fires, capped at 1000 characters with the same `… [+N chars]` marker

This example shows a Stop input with one in-flight shell task and one recurring cron:

```
{
  "session_id": "abc123",
  "transcript_path": "~/.claude/projects/.../00893aaf-19fa-41d2-
8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "Stop",
  "stop_hook_active": true,
  "last_assistant_message": "I've completed the refactoring.
Here's a summary...",
  "background_tasks": [
    {
      "id": "task-001",
      "type": "shell",
      "status": "running",
      "description": "tail logs",
      "command": "tail -f /var/log/syslog"
    }
  ],
  "session_crons": [
    {
      "id": "cron-001",
      "schedule": "0 9 * * 1-5",
      "recurring": true,
      "prompt": "check the build"
    }
  ]
}
```


Stop decision control

`Stop` and `SubagentStop` hooks can control whether Claude continues. In addition to the **JSON output fields** available to all hooks, your hook script can return these event-specific fields:

Field	Description
`decision`	`"block"` prevents Claude from stopping. Omit to allow Claude to stop
`reason`	Required when `decision` is `"block"`. Tells Claude why it should continue
`hookSpecificOutput.additionalContext`	Non-error feedback for Claude. The conversation continues so Claude can act on it, but unlike `decision: "block"` it is shown in the transcript as hook feedback rather than a hook error

```
{
  "decision": "block",
  "reason": "Must be provided when Claude is blocked from
stopping"
}
```

Use `additionalContext` when the hook is working as designed and giving Claude guidance, such as "run the test suite before finishing". It keeps the conversation going through the same loop protections as `decision: "block"`, namely the `stop_hook_active` input and the 8-consecutive-continuation cap, but the transcript labels it `Stop hook feedback` and no hook error notification is shown:

```
{
  "hookSpecificOutput": {
    "hookEventName": "Stop",
    "additionalContext": "Please run the test suite before
finishing"
  }
}
```

StopFailure

Runs instead of **Stop** when the turn ends due to an API error. Output and exit code are ignored. Use this to log failures, send alerts, or take recovery actions when Claude cannot complete a response due to rate limits, authentication problems, or other API errors.

StopFailure input

In addition to the **common input fields**, StopFailure hooks receive `error`, optional `error_details`, and optional `last_assistant_message`. The `error` field identifies the error type and is used for matcher filtering.

Field	Description
`error`	Error type: `rate_limit`, `overloaded`, `authentication_failed`, `oauth_org_not_allowed`, `billing_error`, `invalid_request`, `model_not_found`, `server_error`, `max_output_tokens`, or `unknown`
`error_details`	Additional details about the error, when available
`last_assistant_mess`	The rendered error text shown in the conversation. Unlike `Stop` and `SubagentStop`,

Field	Description
`age`	where this field holds Claude's conversational output, for `StopFailure` it contains the API error string itself, such as `"API Error: Rate limit reached"`

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "StopFailure",
  "error": "rate_limit",
  "error_details": "429 Too Many Requests",
  "last_assistant_message": "API Error: Rate limit reached"
}
```

StopFailure hooks have no decision control. They run for notification and logging purposes only.

TeammateIdle

Runs when an **agent team** teammate is about to go idle after finishing its turn. Use this to enforce quality gates before a teammate stops working, such as requiring passing lint checks or verifying that output files exist.

When a `TeammateIdle` hook exits with code 2, the teammate receives the stderr message as feedback and continues working instead of going idle. To stop the teammate entirely instead of re-running it, return JSON with `{"continue": false, "stopReason": "..."}`. TeammateIdle hooks do not support matchers and fire on every occurrence.

TeammateIdle input

In addition to the **common input fields**, TeammateIdle hooks receive `teammate_name` and `team_name`.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "TeammateIdle",
  "teammate_name": "researcher",
  "team_name": "session-a1b2c3d4"
}
```

Field	Description
`teammate_name`	Name of the teammate that is about to go idle
`team_name`	Deprecated. Session-derived team name; will be removed in a future release

TeammateIdle decision control

TeammateIdle hooks support two ways to control teammate behavior:

- **Exit code 2**: the teammate receives the stderr message as feedback and continues working instead of going idle.
- **JSON** `{"continue": false, "stopReason": "..."}`: stops the teammate entirely, matching `Stop` hook behavior. The `stopReason` is shown to the user.

This example checks that a build artifact exists before allowing a teammate to go idle:

```
#!/bin/bash

if [ ! -f "./dist/output.js" ]; then
  echo "Build artifact missing. Run the build before stopping."
>&2
  exit 2
fi

exit 0
```

ConfigChange

Runs when a configuration file changes during a session. Use this to audit settings changes, enforce security policies, or block unauthorized modifications to configuration files.

ConfigChange hooks fire for changes to settings files, managed policy settings, and skill files. The `source` field in the input tells you which type of configuration changed, and the optional `file_path` field provides the path to the changed file.

The matcher filters on the configuration source:

Matcher	When it fires
`user_settings`	`~/.claude/settings.json` changes
`project_settings`	`.claude/settings.json` changes
`local_settings`	`.claude/settings.local.json` changes
`policy_settings`	Managed policy settings change
`skills`	A skill file in `.claude/skills/` changes

This example logs all configuration changes for security auditing:

```
{
  "hooks": {
    "ConfigChange": [
      {
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/audit-config-change.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

ConfigChange input

In addition to the **common input fields**, ConfigChange hooks receive `source` and optionally `file_path`. The `source` field indicates which configuration type changed, and `file_path` provides the path to the specific file that was modified.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "ConfigChange",
  "source": "project_settings",
  "file_path": "/Users/.../my-project/.claude/settings.json"
}
```

ConfigChange decision control

ConfigChange hooks can block configuration changes from taking effect. Use exit code 2 or a JSON `decision` to prevent the change. When blocked, the new settings are not applied to the running session.

Field	Description
`decision`	`"block"` prevents the configuration change from being applied. Omit to allow the change
`reason`	Explanation shown to the user when `decision` is `"block"`

```
{
  "decision": "block",
  "reason": "Configuration changes to project settings require
admin approval"
}
```

`policy_settings` changes cannot be blocked. Hooks still fire for `policy_settings` sources, so you can use them for audit logging, but any blocking decision is ignored. This ensures enterprise-managed settings always take effect.

CwdChanged

Runs when the working directory changes during a session, for example when Claude executes a `cd` command. Use this to react to directory changes: reload environment variables, activate project-specific toolchains, or run setup scripts automatically. Pairs with **FileChanged** for tools like **direnv** that manage per-directory environment.

CwdChanged hooks have access to `CLAUDE_ENV_FILE`. Variables written to that file persist into subsequent Bash commands for the session, just as in **SessionStart hooks**.

CwdChanged does not support matchers and fires on every directory change.

CwdChanged input

In addition to the **common input fields**, CwdChanged hooks receive `old_cwd` and `new_cwd`.

```
{
  "session_id": "abc123",
  "transcript_path":
"/Users/.../.claude/projects/.../transcript.jsonl",
  "cwd": "/Users/my-project/src",
  "hook_event_name": "CwdChanged",
  "old_cwd": "/Users/my-project",
  "new_cwd": "/Users/my-project/src"
}
```

CwdChanged output

In addition to the **JSON output fields** available to all hooks, CwdChanged hooks can return `watchPaths` to dynamically set which file paths **FileChanged** watches:

Field	Description
`watchPaths`	Array of absolute paths. Replaces the current dynamic watch list (paths from your `matcher` configuration are always watched). Returning an empty array clears the dynamic list, which is typical when entering a new directory

CwdChanged hooks have no decision control. They cannot block the directory change.

FileChanged

Runs when a watched file changes on disk. Useful for reloading environment variables when project configuration files are modified.

The `matcher` for this event serves two roles:

- **Build the watch list**: the value is split on `|` and each segment is registered as a literal filename in the working directory, so `".envrc|.env"` watches exactly those two files. Regex patterns are not useful here: a value like `^\.env` would watch a file literally named `^\.env`.
- **Filter which hooks run**: when a watched file changes, the same value filters which hook groups run using the standard **matcher rules** against the changed file's basename.

FileChanged hooks have access to `CLAUDE_ENV_FILE`. Variables written to that file persist into subsequent Bash commands for the session, just as in **SessionStart hooks**.

FileChanged input

In addition to the **common input fields**, FileChanged hooks receive `file_path` and `event`.

Field	Description
`file_path`	Absolute path to the file that changed
`event`	What happened: `"change"` (file modified), `"add"` (file created), or `"unlink"` (file deleted)

```
{
  "session_id": "abc123",
  "transcript_path":
"/Users/.../.claude/projects/.../transcript.jsonl",
  "cwd": "/Users/my-project",
  "hook_event_name": "FileChanged",
  "file_path": "/Users/my-project/.envrc",
  "event": "change"
}
```

FileChanged output

In addition to the **JSON output fields** available to all hooks, FileChanged hooks can return `watchPaths` to dynamically update which file paths are watched:

Field	Description
`watchPaths`	Array of absolute paths. Replaces the current dynamic watch list (paths from your `matcher` configuration are always watched). Use this when your hook script discovers additional files to watch based on the changed file

FileChanged hooks have no decision control. They cannot block the file change from occurring.

WorktreeCreate

When you run `claude --worktree` or a **subagent uses** `isolation: "worktree"`, Claude Code creates an isolated working copy using `git worktree`. If you configure a WorktreeCreate hook, it replaces the default git behavior, letting you use a different version control system like SVN, Perforce, or Mercurial.

Because the hook replaces the default behavior entirely, `.worktreeinclude` is not processed. If you need to copy local configuration files like `.env` into the new worktree, do it inside your hook script.

The hook must return the absolute path to the created worktree directory. Claude Code uses this path as the working directory for the isolated session. Command hooks print it on stdout; HTTP hooks return it via `hookSpecificOutput.worktreePath`.

This example creates an SVN working copy and prints the path for Claude Code to use. Replace the repository URL with your own:

```
{
  "hooks": {
    "WorktreeCreate": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'NAME=$(jq -r .name);
DIR=\"$HOME/.claude/worktrees/$NAME\"; svn checkout
https://svn.example.com/repo/trunk \"$DIR\" >&2 && echo
\"$DIR\"'"
          }
        ]
      }
    ]
  }
}
```

The hook reads the worktree `name` from the JSON input on stdin, checks out a fresh copy into a new directory, and prints the directory path. The `echo` on the last line is what Claude Code reads as the worktree path. Redirect any other output to stderr so it doesn't interfere with the path.

WorktreeCreate input

In addition to the **common input fields**, WorktreeCreate hooks receive the `name` field. This is a slug identifier for the new worktree, either specified by the user or auto-generated (for example, `bold-oak-a3f2`).

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "WorktreeCreate",
  "name": "feature-auth"
}
```

WorktreeCreate output

WorktreeCreate hooks do not use the standard allow/block decision model. Instead, the hook's success or failure determines the outcome. The hook must return the absolute path to the created

worktree directory:

- **Command hooks** (`type: "command"`): print the path on stdout.
- **HTTP hooks** (`type: "http"`): return `{ "hookSpecificOutput": { "hookEventName": "WorktreeCreate", "worktreePath": "/absolute/path" } }` in the response body.

If the hook fails or produces no path, worktree creation fails with an error.

WorktreeRemove

The cleanup counterpart to **WorktreeCreate**. This hook fires when a worktree is being removed, either when you exit a `--worktree` session and choose to remove it, or when a subagent with `isolation: "worktree"` finishes. For git-based worktrees, Claude handles cleanup automatically with `git worktree remove` . If you configured a WorktreeCreate hook for a non-git version control system, pair it with a WorktreeRemove hook to handle cleanup. Without one, the worktree directory is left on disk.

Claude Code passes the path returned by WorktreeCreate as `worktree_path` in the hook input. This example reads that path and removes the directory:

```
{
  "hooks": {
    "WorktreeRemove": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'jq -r .worktree_path | xargs rm
-rf'"
          }
        ]
      }
    ]
  }
}
```

WorktreeRemove input

In addition to the **common input fields**, WorktreeRemove hooks receive the `worktree_path` field, which is the absolute path to the worktree being removed.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "WorktreeRemove",
  "worktree_path": "/Users/.../my-
project/.claude/worktrees/feature-auth"
}
```

WorktreeRemove hooks have no decision control. They cannot block worktree removal but can perform cleanup tasks like removing version control state or archiving changes. Hook failures are logged in debug mode only.

PreCompact

Runs before Claude Code is about to run a compact operation.

The matcher value indicates whether compaction was triggered manually or automatically:

Matcher	When it fires
`manual`	`/compact`
`auto`	Auto-compact when the context window is full

Exit with code 2 to block compaction. For a manual `/compact`, the stderr message is shown to the user. You can also block by returning JSON with `"decision": "block"`.

Blocking automatic compaction has different effects depending on when it fires. If compaction was triggered proactively before the context limit, Claude Code skips it and the conversation continues uncompacted. If compaction was triggered to recover from a context-limit error already returned by the API, the underlying error surfaces and the current request fails.

PreCompact input

In addition to the **common input fields**, PreCompact hooks receive `trigger` and `custom_instructions`. For `manual`, `custom_instructions` contains what the user passes into `/compact`. For `auto`, `custom_instructions` is empty.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "PreCompact",
  "trigger": "manual",
  "custom_instructions": ""
}
```

PostCompact

Runs after Claude Code completes a compact operation. Use this event to react to the new compacted state, for example to log the generated summary or update external state.

The same matcher values apply as for `PreCompact`:

Matcher	When it fires
`manual`	After `/compact`
`auto`	After auto-compact when the context window is full

PostCompact input

In addition to the **common input fields**, PostCompact hooks receive `trigger` and `compact_summary`. The `compact_summary` field contains the conversation summary generated by the compact operation.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "PostCompact",
  "trigger": "manual",
  "compact_summary": "Summary of the compacted conversation..."
}
```

PostCompact hooks have no decision control. They cannot affect the compaction result but can

perform follow-up tasks.

SessionEnd

Runs when a Claude Code session ends. Useful for cleanup tasks, logging session statistics, or saving session state. Supports matchers to filter by exit reason.

The `reason` field in the hook input indicates why the session ended:

Reason	Description
`clear`	Session cleared with `/clear` command
`resume`	Session switched via interactive `/resume`
`logout`	User logged out
`prompt_input_exit`	User exited while prompt input was visible
`bypass_permissions_disabled`	Bypass permissions mode was disabled
`other`	Other exit reasons

SessionEnd input

In addition to the **common input fields**, SessionEnd hooks receive a `reason` field indicating why the session ended. See the **reason table** above for all values.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "hook_event_name": "SessionEnd",
  "reason": "other"
}
```

SessionEnd hooks have no decision control. They cannot block session termination but can perform cleanup tasks.

SessionEnd hooks have a default timeout of 1.5 seconds. This applies to session exit, `/clear`, and switching sessions via interactive `/resume`. If a hook needs more time, set a per-hook `timeout` in the hook configuration. The overall budget is automatically raised to the highest per-hook timeout

configured in settings files, up to 60 seconds. Timeouts set on plugin-provided hooks do not raise the budget. To override the budget explicitly, set the `CLAUDE_CODE_SESSIONEND_HOOKS_TIMEOUT_MS` environment variable in milliseconds.

```
CLAUDE_CODE_SESSIONEND_HOOKS_TIMEOUT_MS=5000 claude
```

Elicitation

Runs when an MCP server requests user input mid-task. By default, Claude Code shows an interactive dialog for the user to respond. Hooks can intercept this request and respond programmatically, skipping the dialog entirely.

The matcher field matches against the MCP server name.

Elicitation input

In addition to the **common input fields**, Elicitation hooks receive `mcp_server_name`, `message`, and optional `mode`, `url`, `elicitation_id`, and `requested_schema` fields.

For form-mode elicitation (the most common case):

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "Elicitation",
  "mcp_server_name": "my-mcp-server",
  "message": "Please provide your credentials",
  "mode": "form",
  "requested_schema": {
    "type": "object",
    "properties": {
      "username": { "type": "string", "title": "Username" }
    }
  }
}
```

For URL-mode elicitation (browser-based authentication):

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "Elicitation",
  "mcp_server_name": "my-mcp-server",
  "message": "Please authenticate",
  "mode": "url",
  "url": "https://auth.example.com/login"
}
```

Elicitation output

To respond programmatically without showing the dialog, return a JSON object with `hookSpecificOutput`:

```
{
  "hookSpecificOutput": {
    "hookEventName": "Elicitation",
    "action": "accept",
    "content": {
      "username": "alice"
    }
  }
}
```

Field	Values	Description
`action`	`accept`, `decline`, `cancel`	Whether to accept, decline, or cancel the request
`content`	object	Form field values to submit. Only used when `action` is `accept`

Exit code 2 denies the elicitation and shows stderr to the user.

ElicitationResult

Runs after a user responds to an MCP elicitation. Hooks can observe, modify, or block the response

before it is sent back to the MCP server.

The matcher field matches against the MCP server name.

ElicitationResult input

In addition to the **common input fields**, ElicitationResult hooks receive `mcp_server_name`, `action`, and optional `mode`, `elicitation_id`, and `content` fields.

```
{
  "session_id": "abc123",
  "transcript_path": "/Users/.../.claude/projects/.../00893aaf-
19fa-41d2-8238-13269b9b3ca0.jsonl",
  "cwd": "/Users/...",
  "permission_mode": "default",
  "hook_event_name": "ElicitationResult",
  "mcp_server_name": "my-mcp-server",
  "action": "accept",
  "content": { "username": "alice" },
  "mode": "form",
  "elicitation_id": "elicit-123"
}
```

ElicitationResult output

To override the user's response, return a JSON object with `hookSpecificOutput`:

```
{
  "hookSpecificOutput": {
    "hookEventName": "ElicitationResult",
    "action": "decline",
    "content": {}
  }
}
```

Field	Values	Description
`action`	`accept` , `decline` , `cancel`	Overrides the user's action
`content`	object	Overrides form field values. Only meaningful when `action` is `accept`

Exit code 2 blocks the response, changing the effective action to `decline` .

Prompt-based hooks

In addition to command, HTTP, and MCP tool hooks, Claude Code supports prompt-based hooks (`type: "prompt"`) that use an LLM to evaluate whether to allow or block an action, and agent hooks (`type: "agent"`) that spawn an agentic verifier with tool access. Not all events support every hook type.

Events that support all five hook types (`command` , `http` , `mcp_tool` , `prompt` , and `agent`):

- `PermissionDenied`
- `PermissionRequest`
- `PostToolBatch`
- `PostToolUse`
- `PostToolUseFailure`
- `PreToolUse`
- `Stop`
- `SubagentStop`
- `TaskCompleted`
- `TaskCreated`
- `TeammateIdle`
- `UserPromptExpansion`
- `UserPromptSubmit`

Events that support `command` , `http` , and `mcp_tool` hooks but not `prompt` or `agent` :

- `ConfigChange`

- `CwdChanged`
- `Elicitation`
- `ElicitationResult`
- `FileChanged`
- `InstructionsLoaded`
- `Notification`
- `PostCompact`
- `PreCompact`
- `SessionEnd`
- `StopFailure`
- `SubagentStart`
- `WorktreeCreate`
- `WorktreeRemove`

`SessionStart` and `Setup` support `command` and `mcp_tool` hooks. They do not support `http`, `prompt`, or `agent` hooks.

How prompt-based hooks work

Instead of executing a Bash command, prompt-based hooks:

1. Send the hook input and your prompt to a Claude model, Haiku by default
2. The LLM responds with structured JSON containing a decision
3. Claude Code processes the decision automatically

Prompt hook configuration

Set `type` to `"prompt"` and provide a `prompt` string instead of a `command`. Use the `$ARGUMENTS` placeholder to inject the hook's JSON input data into your prompt text. Claude Code sends the combined prompt and input to a fast Claude model, which returns a JSON decision.

This `Stop` hook asks the LLM to evaluate whether all tasks are complete before allowing Claude to finish:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "prompt",
            "prompt": "Evaluate if Claude should stop:
$ARGUMENTS. Check if all tasks are complete."
          }
        ]
      }
    ]
  }
}
```

Field	Required	Description
`type`	yes	Must be `"prompt"`
`prompt`	yes	The prompt text to send to the LLM. Use `$ARGUMENTS` as a placeholder for the hook input JSON. If `$ARGUMENTS` is not present, input JSON is appended to the prompt
`model`	no	Model to use for evaluation. Defaults to a fast model
`timeout`	no	Timeout in seconds. Default: 30
`continueOnBlock`	no	When the prompt returns `ok: false`, feed the reason back to Claude and continue the turn instead of stopping. Default: `false`. Implemented as `continue: true` on the resulting `decision: "block"`. See **Response schema** for per-event behavior

Response schema

The LLM must respond with JSON containing:

```
{
  "ok": true | false,
  "reason": "Explanation for the decision"
}
```

Field	Description
`ok`	`true` to allow. `false` produces a `decision: "block"`. See the per-event behavior below
`reason`	Required when `ok` is `false`. Used as the block reason

What happens on `ok: false` depends on the event:

- `Stop` and `SubagentStop`: the reason is fed back to Claude as its next instruction and the turn continues
- `PreToolUse`: the tool call is denied and the reason is returned to Claude as the tool error, equivalent to a command hook's `permissionDecision: "deny"`
- `PostToolUse`: by default the turn ends and the reason appears in the chat as a warning line. Set `continueOnBlock: true` to feed the reason back to Claude and continue the turn instead
- `PostToolBatch`, `UserPromptSubmit`, and `UserPromptExpansion`: the turn ends and the reason appears as a warning line. These events end the turn on `decision: "block"` regardless of `continue`
- `PostToolUseFailure`, `TaskCreated`, and `TaskCompleted`: the reason is returned to Claude as a tool error, similar to `PreToolUse`
- `TeammateIdle`: by default the teammate stops and the reason appears as a warning line. Set `continueOnBlock: true` to feed the reason back to the teammate and keep it working instead
- `PermissionRequest`: `ok: false` has no effect. To deny an approval from a hook, use a **command hook** returning `hookSpecificOutput.decision.behavior: "deny"`
- `PermissionDenied`: `ok: false` has no effect because the denial already happened. The only output this event reads is `hookSpecificOutput.retry`, which prompt and agent hooks cannot set. They run on this event, but their output is discarded. Use a **command hook** to return `retry`

If you need finer control on any event, use a **command hook** with the per-event fields described in **Decision control**.

Example: Multi-criteria Stop hook

This `Stop` hook uses a detailed prompt to check three conditions before allowing Claude to stop. If `"ok"` is `false`, Claude continues working with the provided reason as its next instruction. `SubagentStop` hooks use the same format to evaluate whether a **subagent** should stop:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "prompt",
            "prompt": "You are evaluating whether Claude should
stop working. Context: $ARGUMENTS\n\nAnalyze the conversation and
determine if:\n1. All user-requested tasks are complete\n2. Any
errors need to be addressed\n3. Follow-up work is
needed\n\nRespond with JSON: {\"ok\": true} to allow stopping, or
{\"ok\": false, \"reason\": \"your explanation\"} to continue
working.",
            "timeout": 30
          }
        ]
      }
    ]
  }
}
```

Agent-based hooks

> ⚠ Agent hooks are experimental. Behavior and configuration may change in future releases. For production workflows, prefer **command hooks**.

Agent-based hooks (`type: "agent"`) are like prompt-based hooks but with multi-turn tool access. Instead of a single LLM call, an agent hook spawns a subagent that can read files, search code, and inspect the codebase to verify conditions. Agent hooks support the same events as prompt-based hooks.

How agent hooks work

When an agent hook fires:

1. Claude Code spawns a subagent with your prompt and the hook's JSON input
2. The subagent can use tools like Read, Grep, and Glob to investigate
3. After up to 50 turns, the subagent returns a structured `{ "ok": true/false }` decision

4. Claude Code processes the decision the same way as a prompt hook

Agent hooks are useful when verification requires inspecting actual files or test output, not just evaluating the hook input data alone.

Agent hook configuration

Set `type` to `"agent"` and provide a `prompt` string. The configuration fields are the same as **prompt hooks**, with a longer default timeout:

Field	Required	Description
`type`	yes	Must be `"agent"`
`prompt`	yes	Prompt describing what to verify. Use `$ARGUMENTS` as a placeholder for the hook input JSON
`model`	no	Model to use. Defaults to a fast model
`timeout`	no	Timeout in seconds. Default: 60

The response schema is the same as prompt hooks: `{ "ok": true }` to allow or `{ "ok": false, "reason": "..." }` to block.

This `Stop` hook verifies that all unit tests pass before allowing Claude to finish:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "agent",
            "prompt": "Verify that all unit tests pass. Run the
test suite and check the results. $ARGUMENTS",
            "timeout": 120
          }
        ]
      }
    ]
  }
}
```

Run hooks in the background

By default, hooks block Claude's execution until they complete. For long-running tasks like deployments, test suites, or external API calls, set `"async": true` to run the hook in the background while Claude continues working. Async hooks cannot block or control Claude's behavior: response fields like `decision`, `permissionDecision`, and `continue` have no effect, because the action they would have controlled has already completed.

Configure an async hook

Add `"async": true` to a command hook's configuration to run it in the background without blocking Claude. This field is only available on `type: "command"` hooks.

This hook runs a test script after every `Write` tool call. Claude continues working immediately while `run-tests.sh` executes for up to 120 seconds. When the script finishes, its output is delivered on the next conversation turn:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/run-tests.sh",
            "async": true,
            "timeout": 120
          }
        ]
      }
    ]
  }
}
```

The `timeout` field sets the maximum time in seconds for the background process. If not specified, async hooks use the same 10-minute default as sync hooks.

How async hooks execute

When an async hook fires, Claude Code starts the hook process and immediately continues without waiting for it to finish. The hook receives the same JSON input via stdin as a synchronous hook.

After the background process exits, if the hook produced a JSON response with an `additionalContext` field, that content is delivered to Claude as context on the next conversation turn. A `systemMessage` field is shown to you, not to Claude.

Async hook completion notifications are suppressed by default. To see them, enable verbose mode with `Ctrl+O` or start Claude Code with `--verbose`.

Example: run tests after file changes

This hook starts a test suite in the background whenever Claude writes a file, then reports the results back to Claude when the tests finish. Save this script to `.claude/hooks/run-tests-async.sh` in your project and make it executable with `chmod +x`:

```
#!/bin/bash
# run-tests-async.sh

# Read hook input from stdin
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path //
empty')

# Only run tests for source files
if [[ "$FILE_PATH" != *.ts && "$FILE_PATH" != *.js ]]; then
  exit 0
fi

# Run tests and report results to Claude via additionalContext
RESULT=$(npm test 2>&1)
EXIT_CODE=$?

if [ $EXIT_CODE -eq 0 ]; then
  MSG="Tests passed after editing $FILE_PATH"
else
  MSG="Tests failed after editing $FILE_PATH: $RESULT"
fi
jq -nc --arg msg "$MSG" '{hookSpecificOutput: {hookEventName:
"PostToolUse", additionalContext: $msg}}'
```

Then add this configuration to `.claude/settings.json` in your project root. The `async: true` flag lets Claude keep working while tests run:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/run-
tests-async.sh",
            "args": [],
            "async": true,
            "timeout": 300
          }
        ]
      }
    ]
  }
}
```

Limitations

Async hooks have several constraints compared to synchronous hooks:

- Only `type: "command"` hooks support `async`. Prompt-based hooks cannot run asynchronously.
- Async hooks cannot block tool calls or return decisions. By the time the hook completes, the triggering action has already proceeded.
- Hook output is delivered on the next conversation turn. If the session is idle, the response waits until the next user interaction. Exception: an `asyncRewake` hook that exits with code 2 wakes Claude immediately even when the session is idle.
- Each execution creates a separate background process. There is no deduplication across multiple firings of the same async hook.

Security considerations

Disclaimer

Command hooks run with your system user's full permissions.

> ⚠ Command hooks execute shell commands with your full user permissions. They can modify, delete, or access any files your user account can access. Review and test all hook commands before adding them to your configuration.

Security best practices

Keep these practices in mind when writing hooks:

- **Validate and sanitize inputs**: never trust input data blindly
- **Always quote shell variables**: use `"$VAR"` not `$VAR`
- **Block path traversal**: check for `..` in file paths
- **Use absolute paths**: specify full paths for scripts. In exec form, use `${CLAUDE_PROJECT_DIR}` and the path needs no quoting. In shell form, wrap it in double quotes
- **Skip sensitive files**: avoid `.env`, `.git/`, keys, etc.

Windows PowerShell tool

On Windows, you can run individual hooks in PowerShell by setting `"shell": "powershell"` on a command hook. Hooks spawn PowerShell directly, so this works regardless of whether `CLAUDE_CODE_USE_POWERSHELL_TOOL` is set. Claude Code auto-detects `pwsh.exe` (PowerShell 7+) with a fallback to `powershell.exe` (5.1).

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "shell": "powershell",
            "command": "Write-Host 'File written'"
          }
        ]
      }
    ]
  }
}
```

Debug hooks

Hook execution details, including which hooks matched, their exit codes, and full stdout and stderr, are written to the debug log file. Start Claude Code with `claude --debug-file <path>` to write the log to a known location, or run `claude --debug` and read the log at `~/.claude/debug/<session-id>.txt`. The `--debug` flag does not print to the terminal.

```
[DEBUG] Executing hooks for PostToolUse:Write
[DEBUG] Found 1 hook commands to execute
[DEBUG] Executing hook command: <Your command> with timeout
600000ms
[DEBUG] Hook command completed with status 0: <Your stdout>
```

For more granular hook matching details, set `CLAUDE_CODE_DEBUG_LOG_LEVEL=verbose` to see additional log lines such as hook matcher counts and query matching.

For troubleshooting common issues like hooks not firing, Stop hooks that keep blocking, or configuration errors, see **Limitations and troubleshooting** in the guide. For a broader diagnostic walkthrough covering `/context`, `/doctor`, and settings precedence, see **Debug your config**.